

3D Printer Pick'n'Place

Jaiveer Safri, Nikolaos Mitsokapas, Rahul Jo-Shua Thanasekaran,
Nevain Dewage

4th Year Project Final Report

Department of Electronic and Electrical Engineering

UCL

Supervisor: Prof. Thomas Gilbert

May 1, 2026

I have read and understood UCL's and the Department's statements and guidelines concerning plagiarism.

I declare that all material described in this report is my own work except where explicitly and individually indicated in the text. This includes ideas described in the text, figures and computer programs.

This report contains 85 pages (excluding this page) and 11,984 words.

Signed:	<u><i>Jaiveer Safri</i></u> (Student)	Date: <u>01 May 2026</u>
Signed:	<u><i>RJT</i></u> (Student)	Date: <u>01 May 2026</u>
Signed:	<u><i>Nevain Dewage</i></u> (Student)	Date: <u>01 May 2026</u>
Signed:	<u><i>Nikolaos</i></u> (Student)	Date: <u>01 May 2026</u>

Contents

1	Executive Summary	5
2	Introduction and Literature Review	6
2.1	Literature Review: Tool-Changing Architectures for FDM Systems	6
2.1.1	Magnetic and Passive Kinematic Couplings	6
2.1.2	Gravity-Held and Pin-Bushing Couplings	7
2.1.3	Active vs Passive Docking Gantry	7
2.2	Literature Review: End-effector Design and Control	8
2.2.1	Comparing Types of End-effectors	8
2.2.2	Design Principles for the Two-Finger Parallel Gripper	9
2.2.3	Encoder-Based PID Control of the Wrist Yaw Joint	10
2.3	Literature Review: Machine Vision System	10
2.3.1	Rolling vs Global Shutter Mechanisms	10
2.3.2	Camera Options and Selection Criteria	11
2.3.3	Transfer Learning in Convolutional Neural Networks	12
2.4	Literature Review: Software Pipeline and Printer Control	13
2.4.1	Geometry Extraction from CAD Systems	13
2.4.2	G-code and Printer Firmware Execution	13
2.5	Project Goals and Objectives	14
3	Theoretical and Mathematical Background	15
3.1	Active Docking System Design Considerations	15
3.1.1	Static Equilibrium and Kinematic Constraint	15
3.1.2	Actuation Kinematics	15
3.2	End-effector	15
3.2.1	Gear Design	15
3.2.2	PID Control	19
3.2.3	Magnetic Position Sensing - AS5600	19
3.3	Orientation Derivation Through Mesh Comparison	20
4	Methods, Experimental Setup, and Software Implementation	21
4.1	Tool-changer	21
4.1.1	Mechanical Implementation and Assembly	21
4.1.2	Electronics and Firmware Configuration	22
4.1.3	Component Tray	24
4.1.4	Software Logic and Tool-changing Sequence	24
4.1.5	Experimental Setup for Validation	24
4.1.6	Daughterboard Design	26
4.2	End-effector System Overview	27
4.2.1	Mechanical Design and Assembly	27
4.2.2	Sensing and Feedback Implementation	30
4.2.3	Control System Implementation	30
4.2.4	Controller Tuning	31
4.2.5	Wrist Servo Motor Calibration	31
4.2.6	Software Architecture	33
4.3	Machine Vision System Implementation	34
4.3.1	System Design and Hardware Selection	34
4.3.2	Experimental Setup	34
4.3.3	Dataset Collection and Preparation	36

4.3.4	Model Architecture and Training Strategy	37
4.3.5	Model Deployment	39
4.3.6	Evaluation and Inference	39
4.4	Software and Firmware Implementation	40
4.4.1	CAD Preparation and Embedded Component Extraction	40
4.4.2	Slicer Profile Design	42
4.4.3	G-code Streaming and Embedding Implementation	43
5	Results, Analysis, and Discussion	46
5.1	Tool-Changer System Operational Success	46
5.1.1	Dynamic Rigidity, Resonance, and Wear Analysis	46
5.1.2	Swap Efficiency and Thermal Stability	46
5.1.3	Positional Repeatability and Layer Continuity	47
5.1.4	Operational Considerations and Slicer Mitigation	47
5.2	End-effector Results, Analysis, and Discussion	48
5.2.1	Calibration Quality and Residual Analysis	48
5.2.2	Closed-loop Wrist Control Performance	48
5.3	Machine Vision Results and Analysis	51
5.3.1	Model Performance	51
5.3.2	Training Behaviour	52
5.3.3	System-Level Evaluation	53
5.3.4	Model Interpretability	55
5.4	Reliability of the Automated Component Embedding Pipeline	55
5.4.1	Accuracy of Orientation Estimation	55
5.4.2	Computational Efficiency of the Extraction Pipeline	56
5.4.3	Embedding Layer Adhesion	57
5.5	Repeatability of the Integrated Pick-and-Place Procedure	58
6	Conclusion and Future Work	60
6.1	Toolchanger Subsystem Conclusion	60
6.1.1	Future Work and System Optimisations	60
6.2	End-effector Subsystem Conclusion	60
6.2.1	Future Work	60
6.3	Machine Vision System Conclusion	61
6.3.1	Future Work	61
6.4	Software and Firmware Pipeline Conclusion	61
6.4.1	Future Work	62
7	References	63
8	Team Contribution	65
9	Appendix	66
9.1	Appendix A: Machine Vision Implementation Code	66
9.2	Appendix B: Embedded Component Extraction Code	70
9.3	Appendix C: G-code Streaming Implementation Script	76
9.4	Appendix D: End-effector Arduino Control Code	81
9.5	Appendix E: RAMBo Daughterboard Layout	86
9.6	Appendix F: Snippets of modified marlin code	86
9.6.1	1. Advanced Gripper Yaw Control (PID Loop)	86
9.6.2	2. RAMBo Pin Mapping for Additional U-Axis	88

9.6.3	3. Configuration Limits and Kinematics	88
9.6.4	4. Advanced Configuration for Gripper Servos	89

1 Executive Summary

Additive Manufacturing (AM), specifically Fused Deposition Modelling (FDM), is increasingly used for functional parts, but remains mostly limited to thermoplastics. Manually inserting components like magnets and threaded fasteners adds functionality but creates a significant production bottleneck. This project developed and validated a fully automated Pick-and-Place (PnP) system for a modified LulzBot TAZ 5 3D printer. By synchronising an active tool-changer, precision gripper, computer vision, and a custom software pipeline, the system eliminates manual intervention during hybrid assembly fabrication.

The solution was delivered through four integrated work streams:

Tool-Changer: An independent, active docking gantry (U-axis) was developed using a gravity-held, pin-and-bushing kinematic coupling to facilitate reliable toolhead swapping on a Cartesian “bed-slinger” architecture.

End-effector: A lightweight, rack-and-pinion-controlled parallel gripper was designed with 0.7 mm fingertips to accommodate small cavities, ensuring precision via closed-loop PID yaw control, aided by an AS5600 magnetic rotary encoder, to correct for non-linearities in 3D-printed gears.

Machine Vision: A Raspberry Pi-based global shutter camera system was implemented to provide real-time verification. A MobileNet V2 transfer-learning model was trained to classify nut-placement success with high confidence.

Software Pipeline: A custom Python-based digital pipeline was engineered to extract component pose data directly from user-input files and stream modified G-code instructions to the printer.

Key Results and Performance

The system’s performance was rigorously validated through empirical testing, achieving the following outcomes:

- **Operational Reliability and Kinematic Precision:** The tool-changing mechanism achieved a 100% success rate during a continuous 50-cycle tool-changing stress test, while the active gantry maintained positional repeatability of ± 0.05 mm, ensuring seamless layer continuity and no visible shifting in printed parts.
- **Control Accuracy:** The closed-loop wrist positioning achieved a steady-state error of 0.5° , well within the required tolerances for hardware insertion.
- **Verification Performance:** The machine vision system achieved a validation accuracy of 80.43%, reliably identifying misaligned or partially inserted components.
- **Computational Efficiency:** The automated extraction script processed embedded components in an average of 10 ms per object, ensuring a responsive workflow.

This project proves that high-fidelity, automated hybrid manufacturing is achievable on low-cost, open-source printer platforms. The integration of active electromechanical couplings and intelligent verification establishes a robust framework for the scalable production of multi-functional, 3D-printed assemblies.

2 Introduction and Literature Review

Fused Deposition Modelling (FDM) has evolved into a viable manufacturing method, though its functionality remains limited by the properties of thermoplastic materials. Creating complex assemblies often requires embedding foreign components, such as fasteners or sensors, into the print matrix. Currently, this relies on inefficient, error-prone manual intervention—pausing the printer, inserting the part, and resuming—which limits the scalability of hybrid manufacturing. This project automates the process on a modified LulzBot TAZ 5 3D printer by developing a Pick-and-Place (PnP) system that autonomously pauses, retrieves a component, inserts it, and resumes printing. To achieve this, the project is divided into four integrated work streams:

- **System Architecture:** The implementation of a toolchanger system for a Cartesian printer with the purpose of allowing the printer to automatically and reliably swap between the print head and an end effector during its operation.
- **End-effector:** The implementation of a gripper end-effector capable of handling small components with high precision.
- **Machine Vision:** The implementation of an optical verification system of object placements using a transfer-learning approach.
- **Software and Control:** The implementation of a digital pipeline to convert computer-aided design (CAD) models into executable instructions by parsing 3MF files, generating modified G-code and coordinating PnP operations.

2.1 Literature Review: Tool-Changing Architectures for FDM Systems

Automated embedding during an active FDM print cycle requires a highly repeatable end-effector swapping mechanism. While commercial systems like the E3D ToolChanger are optimised for CoreXY architectures [1], implementing a tool-changer on a Cartesian “bed-slinger” like the LulzBot TAZ 5 introduces unique mass and vibration constraints [2]. Existing paradigms were evaluated to identify a solution that minimises the mass of moving parts while ensuring sub-millimetre repeatability.

2.1.1 Magnetic and Passive Kinematic Couplings

Initial investigations into passive magnetic couplings (e.g., Lineux system [3]) were rejected following a force vector analysis of the LulzBot TAZ 5 extruder assembly. Given the high mass ($m = 0.675$ kg) and a standard travel acceleration ($a_x = 3$ m/s²), the lateral inertial force (F_i) and gravitational load (F_g) are:

$$F_i = m \cdot a_x = 0.675 \cdot 3 = 2.025 \text{ N} \quad (1)$$

$$F_g = m \cdot g = 0.675 \cdot 9.81 = 6.62 \text{ N} \quad (2)$$

The total shear force vector (F_{lat}) acting on the toolhead interface is the resultant of these orthogonal components:

$$F_{lat} = \sqrt{F_g^2 + F_i^2} = \sqrt{6.62^2 + 2.025^2} \approx 6.92 \text{ N} \quad (3)$$

To prevent slippage under dynamic loads, a safety factor (S.F.) of 2.0 and a lower friction bound ($\mu_s = 0.15$) for nickel-plated magnets were applied to determine the minimum required pull force (F_{pull_min}) [4]:

$$F_{pull_min} = \frac{F_{lat} \cdot \text{S.F.}}{\mu_s} = \frac{6.92 \cdot 2.0}{0.15} \approx 92.27 \text{ N} \quad (4)$$

While a 3×2 N52 magnet array satisfies this requirement with a total pull force (F_{pull_total}) of 213.6 N, the lateral force required to mechanically disengage the array for undocking (F_{undock}) is governed by the upper friction bound (0.25):

$$F_{undock} = \mu_s \cdot F_{pull_total} = 0.25 \cdot 213.6 = 53.4 \text{ N} \quad (5)$$

Comparative analysis with the TAZ 5 NEMA 17 X-axis motor [5] indicates that at a travel speed of 200 mm/s (300 RPM), the dynamic torque (τ_{dyn}) drops to 0.25 Nm. The pitch radius (r_p) is derived from the pulley geometry [6], where the belt pitch is $p = 2$ mm, and the number of teeth is $N = 20$:

$$D_p = \frac{p \cdot N}{\pi} = \frac{2 \text{ mm} \cdot 20}{\pi} \approx 12.732 \text{ mm} \quad (6)$$

$$r_p = \frac{D_p}{2} \approx 6.366 \text{ mm} = 0.006366 \text{ m} \quad (7)$$

The maximum linear force (F_{max}) generated via the pulley is calculated as:

$$F_{max} = \frac{\tau_{dyn}}{r_p} = \frac{0.25 \text{ Nm}}{0.006366 \text{ m}} \approx 39.27 \text{ N} \quad (8)$$

Since $F_{undock} > F_{max}$, the motor would invariably stall during the undocking sequence. This mathematical rejection substantiates the pivot from magnetic retention toward an active, pin-and-bushing exact-constraint design.

2.1.2 Gravity-Held and Pin-Bushing Couplings

Alternative mechanical retention strategies, such as the Stealthchanger [7], utilise gravity-held kinematic couplings. The interface consists of stainless steel dowel pins, conforming to ISO 2338 [8], and brass bushings, adhering to strict ISO 286 [9]: H8 (+0.000 / -0.018 mm) for pins and F7 (+0.010 / +0.022 mm) for bushings. Tapered dowel ends enable passive self-alignment, compensating for minor positional inaccuracies during docking.

For the TAZ 5, the high mass of the extruder assembly serves as a mechanical strength in this architecture, providing superior seating stability and rotational constraint compared to horizontal magnetic attraction.

2.1.3 Active vs Passive Docking Gantries

Docking station placement is governed by the printer's kinematic architecture. CoreXY systems facilitate rear-mounted passive docks; however, the Z-axis translation of Cartesian bed-slingers restricts fixed docks to the lateral extremities or overhead, both of which reduce the effective print volume. An active docking system extends the machine footprint to preserve the original print volume, allowing the gantry to move independently of the primary printer axes.

2.2 Literature Review: End-effector Design and Control

2.2.1 Comparing Types of End-effectors

Robotic pick-and-place systems typically employ electromagnetic grippers, vacuum or pneumatic suction, or electromechanical finger grippers. Gripper design cannot be separated from the part characteristics, handling phases, and releasing strategy [10], which proves decisive here, where releasing into a tight cavity is as critical as pickup. Electromagnetic grippers offer simplicity and short cycle times [11], but operate only on ferromagnetic workpieces, immediately disqualifying them for a system handling both steel nuts and non-ferromagnetic permanent magnets. Residual magnetisation after de-energisation can also retain the workpiece at the pole face [10], unacceptable for placement into precisely dimensioned cavities.

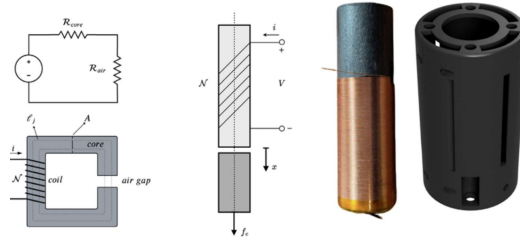


Figure 1: Operating principle of an electromagnetic gripper. The left diagrams show the equivalent magnetic circuit, in which reluctances of the core (R_{core}) and air gap (R_{air}) govern the flux produced by a coil of N turns driven by current i through a core of cross-sectional area A and path length l . The centre schematic illustrates the resulting attraction force f_e generated at the air gap of length x when a voltage V is applied. The right-hand images show a representative solenoid-type electromagnetic end-effector, comprising a wound copper coil within a ferromagnetic housing. The holding force is entirely dependent on maintaining electrical excitation and drops sharply with increasing air gap; release requires de-energisation, which may leave residual magnetisation in the workpiece [11].

Vacuum and suction grippers offer low cycle times and simple control [10], but require a pump or compressed air supply incompatible with the toolhead mass budget. A clean release requires active venting [10], and the curved surfaces of cylindrical magnets prevent reliable cup sealing at these scales.

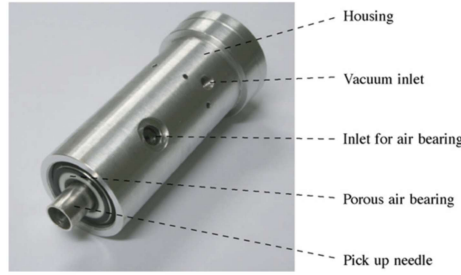


Figure 2: A miniature vacuum-based pick-up end-effector shown with key components labelled [12]: housing, vacuum inlet, air bearing inlet, porous air bearing surface, and pick-up needle. Suction is applied through the vacuum inlet and transmitted to the needle tip, which contacts the component to be picked. The porous air bearing reduces friction and stabilises the needle during approach. Reliable gripping depends on achieving an airtight seal between the needle tip and the component surface; components with irregular, curved, or porous surfaces present significant sealing challenges. An active venting or blow-off mechanism is required for controlled release [10].

The electromechanical two-finger parallel gripper was therefore selected as the only approach offering material independence, deterministic release, and self-centring jaw geometry compatible with both component types, without external infrastructure [11].

2.2.2 Design Principles for the Two-Finger Parallel Gripper

The SCHUNK EGP series, shown in Fig. 3, provides a directly relevant reference, using a pinion-and-rack kinematic to synchronise bilateral jaw motion with stated repeatability of $\pm 0.1 \text{ mm}$ [13]. Its 24 V drive and precision-machined guides are incompatible with the tool-head constraints, but the symmetric rack-and-pinion jaw actuation architecture was identified as directly transferable at a smaller scale.



Figure 3: SCHUNK EGP electric two-finger parallel gripper, used as a reference design during end-effector selection. The gripper uses a rack-and-pinion kinematic to synchronise bilateral jaw motion, ensuring both jaws translate symmetrically about a fixed centreline and producing self-centring behaviour on the gripped workpiece. Roller-bearing jaw guidance provides high repeatability under load. While the industrial specification of this unit, 24 V drive, precision ground rail guides, gripping forces up to 140 N, is incompatible with the mass and integration constraints of a 3D printer toolhead, the core mechanical architecture of symmetric rack-and-pinion jaw actuation was adopted as the basis for the custom gripper developed in this project. Image courtesy of SCHUNK GmbH & Co. KG [13].

FDM manufacturing introduces dimensional variability and anisotropic interlaminar strength

[14]; thin, cantilevered features are particularly susceptible because bending loads act across layer boundaries [15], motivating a compact, stiff finger geometry. Hybrid rigid–soft contact interfaces increase effective contact area through local deformation, improving grasping robustness without sacrificing stiffness [16].

2.2.3 Encoder-Based PID Control of the Wrist Yaw Joint

PID control was selected as the standard feedback strategy for servo positioning [17]. The governing control law is:

$$u(t) = K_p e(t) + K_i \int e(t) dt + K_d \frac{de(t)}{dt} \quad (9)$$

where $u(t)$ is the control output, $e(t)$ is the error between the desired and measured angle, and K_p , K_i , K_d are the proportional, integral, and derivative gains, respectively [17].

The AS5600 magnetic encoder provides absolute, contactless, and frictionless angle measurement [18], offering a resolution of $0.088^\circ/\text{count}$ and eliminating error accumulation during repeated movements. Its specified air gap of $0.5\text{--}3\text{ mm}$ and maximum radial misalignment of 0.25 mm [18] directly constrain the wrist sensor mounting geometry.

2.3 Literature Review: Machine Vision System

2.3.1 Rolling vs Global Shutter Mechanisms

Image sensors in machine vision systems typically operate with either a rolling-shutter or a global-shutter mechanism, which differ in how image data is captured over time. A rolling shutter records an image sequentially, line-by-line across the sensor, meaning different parts of the image correspond to slightly different time instances. This can introduce motion-related distortions such as skewing or stretching when there is relative motion between the camera and the object, as illustrated in Figure 4.

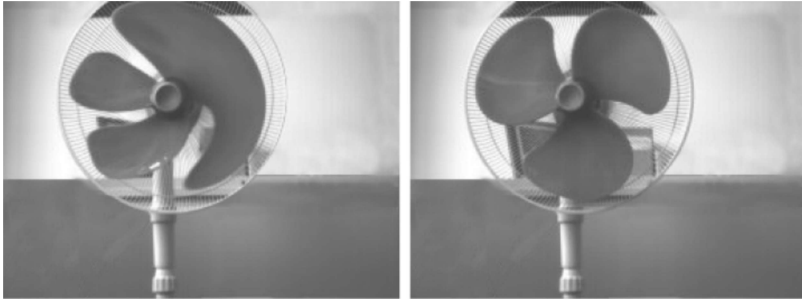


Figure 4: Comparison of shutter mechanisms [19]. The left image shows rolling-shutter distortion, in which the rotating fan blades appear skewed due to sequential image capture. The right image shows a global-shutter capture, in which the fan blades are accurately represented without geometric distortion.

In contrast, a global shutter captures all pixels simultaneously, effectively freezing the scene at a single instant. This eliminates geometric distortion and preserves the true shape and position of objects, making it more suitable for applications involving motion that require accurate spatial representation.

In this project, although the camera and object remain stationary during image capture, a global shutter was selected to ensure maximum image accuracy. Minor vibrations or un-

intended movement can introduce distortion with a rolling shutter, potentially affecting key features used for classification. As precise feature extraction is critical for reliable machine vision performance, a global shutter ensures consistent and accurate image acquisition.

2.3.2 Camera Options and Selection Criteria

Several camera solutions were considered during the initial design phase, as summarised in Table 1 and illustrated in Figure 5. These included industrial machine vision cameras, embedded camera modules, and depth-sensing devices. Selection was guided by key factors including cost, compatibility with embedded processing platforms, image quality, and suitability for machine vision tasks.



Figure 5: Visual comparison of camera systems evaluated during the design phase, highlighting differences in form factor, mounting requirements, and overall integration complexity within an embedded machine vision setup.

Camera	Type	Price (GBP)
Basler ace acA2500-14gc	Industrial GigE camera	£400
NileCAM25 (AR0234)	Embedded USB camera module	£220
Intel RealSense D435	RGB-D depth camera	£240
Arducam IMX296	USB global shutter camera	£150
Cognex In-Sight 7000	Industrial smart vision system	>£1200
Raspberry Pi Global Shutter Camera	SBC camera module	£48

Table 1: Comparison of camera options based on type and approximate cost, illustrating the trade-offs between industrial-grade systems and lower-cost embedded solutions considered for this project.

Industrial cameras such as the Basler Ace and Cognex In-Sight systems offer high precision, robustness, and advanced features, but were unsuitable due to their significantly higher cost and complexity relative to the project requirements. Similarly, depth cameras such as the Intel RealSense D435 provide additional spatial information.

Embedded camera modules, including the Arducam IMX296 and Raspberry Pi Global Shutter Camera, were identified as more suitable due to lower cost and direct compatibility with Raspberry Pi boards. The availability of global-shutter options in this category aligns with the requirements discussed in Section 2.3.1, ensuring distortion-free image capture.

Based on these considerations, the Raspberry Pi Global Shutter Camera was selected as the final solution, providing a balance between cost, imaging performance, and ease of integration while supporting the constraints of an embedded system and enabling a compact machine vision setup.

2.3.3 Transfer Learning in Convolutional Neural Networks

Convolutional Neural Networks (CNNs) are the dominant approach for image classification due to their ability to automatically learn hierarchical feature representations directly from raw pixel data. Early layers typically capture low-level features such as edges and textures, while deeper layers learn more complex and object-specific patterns. This makes CNNs highly effective for visual recognition compared to traditional feature-based methods, which rely on manually engineered descriptors.

However, training CNNs from scratch requires large labelled datasets and significant computational resources, which are often unavailable in practical engineering applications. To address this, transfer learning is commonly employed, where a model pre-trained on a large-scale dataset is adapted to a new task. In this approach, learned feature representations are reused, while only a subset of the network is fine-tuned for the target application.

This project adopts transfer learning to enable efficient training with a relatively small dataset. By leveraging features learned from large-scale datasets, the model can generalise effectively to nut placement classification while reducing training time and computational requirements. This approach also aligns with project time constraints, allowing development to focus on data acquisition, system integration, and performance validation rather than extensive dataset collection and training from scratch.

2.4 Literature Review: Software Pipeline and Printer Control

2.4.1 Geometry Extraction from CAD Systems

Modern AM workflows rely on intermediate file formats to transfer geometric information from CAD systems to slicing software, traditionally achieved using STL (standard tessellation language) files, which represent geometry through tessellated triangular meshes. However, STL-based representations are limited to geometric data and do not encode additional information such as materials, metadata or object hierarchy, motivating the development of more expressive file formats [26].

The 3MF (3D Manufacturing Format) was introduced as an XML-based alternative designed to extend data representation capabilities within additive manufacturing systems. It enables the storage of geometric, material, and metadata information in a single file [26], allowing access to individual components of a model.

The 3MF specification defines a common file structure in which each component is represented as an individual `<object>` element with associated vertex data, and instantiated through `<build>` references. However, it does not standardise the metadata stored, resulting in variation in how different software tools encode and store information. In this work, 3MF files were exported from Autodesk Fusion because they provide a predictable structure for storing component names, enabling embedded components to be identified using naming conventions and separated from the primary print geometry prior to slicing. The extracted vertex data can then be used to compute spatial properties such as mid-point position and orientation.

2.4.2 G-code and Printer Firmware Execution

G-code is a machine-readable instruction set used to translate geometric designs into executable manufacturing operations, defining tool movement through coordinate-based positioning and auxiliary machine commands. Each line of G-code, referred to as a block, contains structured instructions that may specify motion, feedrate, or machine state, enabling precise control over the fabrication process [27].

In AM systems, G-code is interpreted and executed by printer firmware, which translates these instructions into coordinated motion and process control. Firmware typically processes instructions sequentially, ensuring deterministic execution and repeatable toolpaths. However, the capabilities and limitations vary across firmware.

The native firmware used by the TAZ 5, Marlin, has limited support for conditional logic and event-driven behaviour [28]. As a result, advanced operations such as mid-print component embedding require additional coordination beyond standard firmware capabilities. In this work, this limitation is addressed through external control of the instruction stream, enabling synchronisation between the printing process and embedding operations.

2.5 Project Goals and Objectives

Based on the technical constraints identified, the following subsystem objectives were established:

- **Goal 1: Development of an Active Tool-Changing System**
 - **Sub-objective 1.1:** Implement a gravity-reliant kinematic mounting system capable of supporting the toolheads without positional drift or degradation of print quality.
 - **Sub-objective 1.2:** Develop a tool-change sequence with consistent mechanical movements, achieving a 100% success rate over a continuous 50-cycle stress test to validate software state-machine robustness.
- **Goal 2: High-Precision End-Effector Control**
 - **Sub-objective 2.1:** Design a rack-and-pinion gripper with a 0.7 mm tip thickness to provide necessary clearance for small hexagonal nut cavities.
 - **Sub-objective 2.2:** Utilise closed-loop PID control to reduce steady-state angular error to $< 0.05^\circ$, successfully overcoming the 0.65° non-linearities inherent in FDM-manufactured gears.
- **Goal 3: Intelligent Process Verification**
 - **Sub-objective 3.1:** Deploy a global shutter camera system to eliminate motion-induced distortion during mid-print verification cycles.
 - **Sub-objective 3.2:** Achieve a validation accuracy of $\geq 75\%$ for nut-placement classification using a MobileNetV2 transfer-learning architecture.
- **Goal 4: Efficient Digital Pipeline**
 - **Sub-objective 4.1:** Automate the extraction of component pose data from 3MF files with a processing overhead of < 20 ms per component.
 - **Sub-objective 4.2:** Implement a G-code streaming interface capable of synchronised mid-print interventions and automated thermal management pauses.

3 Theoretical and Mathematical Background

3.1 Active Docking System Design Considerations

3.1.1 Static Equilibrium and Kinematic Constraint

The coupling utilises an exact-constraint geometry in which a structural notch bears the primary vertical load ($F_g = 6.62$ N), with two upper ISO 2338 stainless steel pins and brass bushings constraining rotational moments.

The forward-pitching moment (M_{pitch}) generated by the extruder assembly about the notch is calculated using an approximate horizontal centre of mass distance (d_{com}) of 45 mm:

$$M_{pitch} = F_g \cdot d_{com} = 6.62 \cdot 0.045 \approx 0.298 \text{ Nm} \quad (10)$$

To maintain static equilibrium, a countering horizontal reaction force (F_{pull}) is required from the two upper pins, located at a vertical height (h_{pins}) of 50 mm above the pivot:

$$F_{pull} = \frac{M_{pitch}}{h_{pins}} = \frac{0.298}{0.050} = 5.96 \text{ N} \quad (11)$$

This load results in approximately 2.98 N per pin, which is negligible compared to the > 250 MPa yield strength of standard ISO stainless steel. Furthermore, lateral inertial forces during active printing ($F_i = 2.025$ N) generate stresses orders of magnitude below the 15 MPa interlaminar yield strength of the PETG housing. Consequently, the retention geometry operates within a virtually infinite safety margin.

3.1.2 Actuation Kinematics

The independent docking gantry utilises a dual-rack-and-pinion transmission to convert rotational stepper motion into linear translation. Using a Module 2.0 gear profile with a 7-tooth pinion ($D_p = 14$ mm), 1.8° stepper motors ($S_{rev} = 200$), and 1/16 microstepping, the theoretical linear resolution (R_{linear}) is derived as:

$$R_{linear} = \frac{\pi \cdot D_p}{S_{rev} \cdot \mu} = \frac{\pi \cdot 14}{200 \cdot 16} \approx 0.0137 \text{ mm/microstep} \quad (12)$$

This resolution requires a firmware configuration of 72.76 steps/mm. To ensure absolute accuracy and eliminate accumulated error, the gantry is programmed to home against physical endstops after every tool swap.

3.2 End-effector

The design and operation of the gripper subsystem draws on principles from gear design, magnetic sensing, and servo control theory. Each of these is addressed in turn below.

3.2.1 Gear Design

Figure 6 depicts the gripper attached to the gripper mount of the LulzBot TAZ 5 3D printer.

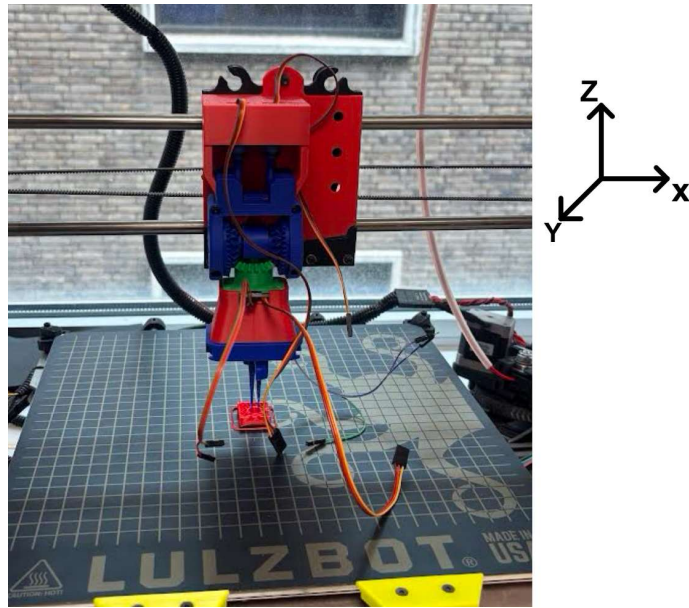


Figure 6: The assembled end-effector mounted to the front plate on the LulzBot TAZ 5 3D printer, shown with the XYZ coordinate frame indicating the printer's axis convention. The wrist assembly, housing the bevel gear transmission and AS5600 encoder, is visible above the gripper fingers. The coordinate frame illustrates the operational context. The printer bed performs Y translation, the extruder carriage performs XZ translation, and the end-effector provides yaw rotation and gripper open/close as the active degrees of freedom.

Initially, the degrees of freedom (DoF) required had to be defined. The LulzBot TAZ 5 printer bed handles Y and the extruder the Z translation, so the end-effector requires yaw rotation, pitch rotation, and gripper open/close. Pitch was de-prioritised given the focus on placement perpendicular to the bed, but can be incorporated if required.

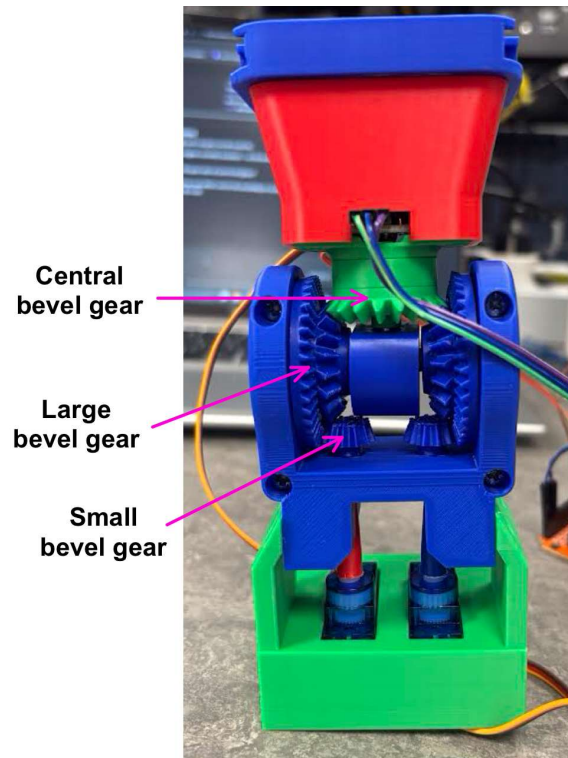


Figure 7: The wrist provides yaw rotation by driving both servos in the same direction, and pitch rotation by driving them in opposing directions via translation of the central bevel gear. The assembly is designed to be compact enough to mount directly to the printer toolhead without exceeding the carriage mass budget, with servo motors positioned symmetrically about the wrist axis to balance the load distribution.

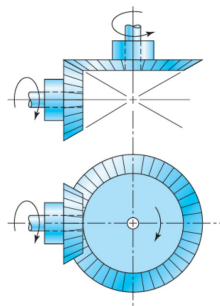


Figure 8: Illustration of the bevel gear meshing principle used in the wrist transmission, showing how rotary motion is redirected between two intersecting shafts. Unlike spur gears, which transmit motion between parallel shafts, bevel gears allow the axis of rotation to be changed, enabling the servo motors, oriented horizontally and symmetrically about the wrist axis, to drive a vertically oriented output shaft. This configuration is central to the wrist design, as it allows both yaw and pitch rotation to be achieved from a compact, symmetric actuator arrangement without requiring the motors to be co-axial with the output.

Gripper open/close motion is provided by a single servo motor via a rack-and-pinion mechanism. Yaw and pitch rotations each require two servo motors. Rotating both in the same direction produces yaw, while opposite directions produce pitch via translation of the central

bevel gear. Two gear types are used. Spur gears, which transmit motion between parallel shafts [29], and bevel gears, which redirect motion between intersecting shafts, as shown in Figs. 7 and 8 [29].

Effective wrist operation requires sufficient torque transmission from the servo motors to the output shaft. Torque is defined as:

$$T = rF \quad (13)$$

where T is torque, r is the radial distance from the axis, and F is the applied force [29]. In gear systems, torque transmission is governed by conservation of power (neglecting losses):

$$P_{in} = P_{out} \Rightarrow T_{in}\omega_{in} = T_{out}\omega_{out} \quad (14)$$

giving the torque ratio between meshing gears as:

$$\frac{T_{out}}{T_{in}} = \frac{N_{out}}{N_{in}} \quad (15)$$

where N represents the number of teeth on each gear [29]. This relationship indicates the torque can be increased or decreased depending on the gear ratio, enabling the system to match motor output characteristics to the mechanical demand of the gripper.

To verify that the available torque is sufficient for reliable nut gripping, the minimum gripping force must be estimated. The gripping force F_g needed to prevent a hexagonal nut from slipping between the two finger faces is governed by the friction at the contact interface:

$$F_{required} = \frac{mg}{\mu n} \quad (16)$$

where m is the nut mass, g is gravitational acceleration, μ is the coefficient of friction at the finger–nut interface, and n is the number of contact faces. For an M6 steel nut, the mass is approximately 3.5 g . FDM-printed PLA sliding against steel exhibits a static coefficient of friction of 0.49–0.67 depending on surface finish and print orientation [30]. Taking a conservative lower bound of $\mu = 0.49$:

$$F_{required} = \frac{0.0035 * 9.81}{0.49 * 2} \approx 0.035 \text{ N} \quad (17)$$

The SG90 servo provides a stall torque of $T_s = 2.5 \text{ kg}\cdot\text{cm} = 0.245 \text{ N}\cdot\text{m}$ at 4.8 V [31]. For the rack-and-pinion drive with pinion pitch radius $r = 15 \text{ mm} = 0.015 \text{ m}$:

$$F_{available} = \frac{T_s}{r} = \frac{0.245}{0.015} \approx 16.3 \text{ N} \quad (18)$$

At 70% transmission efficiency the available gripping force is 11.4 N, giving a safety factor far exceeding requirements, therefore, servo torque is not a limiting constraint. Backlash, the clearance between mating teeth, causes lost motion on direction reversal, introducing a dead zone where input displacement produces no output motion. In FDM gears this is very pronounced due to dimensional variability, degrading positional accuracy. The angular dead zone is given by:

$$\theta_b = \frac{b}{r} \quad (19)$$

where b is the linear backlash and r is the pitch radius. Within this range, small control inputs fail to generate movement until the clearance is taken up.

3.2.2 PID Control

The servo's internal potentiometer loop only regulates the shaft itself. Beyond it, FDM gear backlash introduces a dead zone where input produces no output, compounded by inter-unit servo variability that precludes a single shared open-loop calibration. This means a purely proportional controller would leave a persistent steady-state error. The system therefore employs a PID controller to actively compensate for the discrepancy between the desired and measured wrist angle, shown in Eq. 9.

Ziegler–Nichols tuning assumes a linear plant [32], an assumption violated by backlash. Manual iterative tuning with direct step-response observation is therefore more appropriate [32].

The proportional term drives the wrist toward the target immediately. The integral term accumulates error over time, providing the sustained effort needed to push through the backlash dead zone, and the derivative term damps oscillation arising from gear mesh compliance.

Closed-loop performance is evaluated using three standard metrics [17]: rise time (t_r), the time for the output to travel from 10% to 90% of the step change. Settling time (t_s), the time until the output remains within a $\pm 2\%$ band of the final value. Steady-state error (e_{ss}), the persistent angular difference between commanded and measured angle once the transient decays, which is non-zero under proportional-only control in a backlash-affected system and is eliminated by the integral term.

3.2.3 Magnetic Position Sensing - AS5600

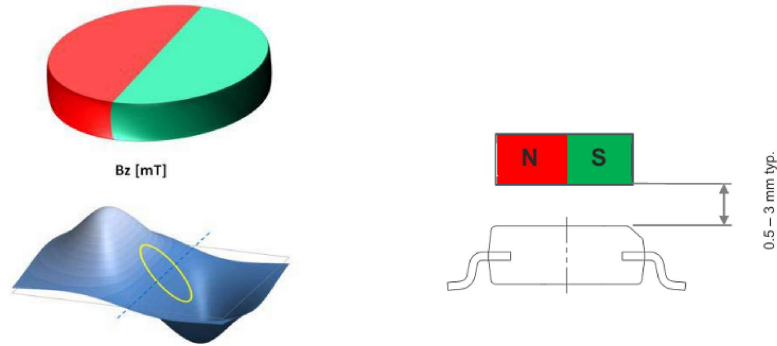


Figure 9: Hall-effect-based magnetic position sensing. Rotation of a diametrically magnetised magnet produces a varying magnetic field (B_z) across the sensor, enabling absolute angular position measurement [33]

Closed-loop wrist control requires direct angular position measurement, achieved using a magnetic rotary sensor based on the Hall effect.

The AS5600 datasheet specifies, a typical air gap of 0.5–3 mm, and a maximum rotational axis misalignment of 0.25 mm from the package centre [33]. These constraints directly govern the wrist sensor mounting geometry, as inaccurate magnet alignment degrades measurement quality and control performance. The AS5600 converts the field orientation into a 12-bit absolute angular position measurement [33], providing the feedback signal for the wrist PID controller.

3.3 Orientation Derivation Through Mesh Comparison

To enable accurate placement of embedded components, the spatial pose of each component must be determined relative to the printer coordinate system. In this system, the nominal pickup configuration is defined as a zero-orientation reference. The component orientation is therefore obtained by comparing a reference mesh, representing this pickup configuration, with the embedded mesh extracted from the user-input CAD model. The required placement is then determined by estimating the rigid transformation that aligns the reference geometry with the embedded geometry.

A rigid transformation preserves shape and size, and can therefore be expressed as a combination of rotations and translations:

$$\mathbf{y}_i = R\mathbf{x}_i + \mathbf{t} \quad (20)$$

where $\mathbf{x}_i$ and $\mathbf{y}_i$ are corresponding points in the original and transformed coordinate frames respectively, R is a 3-dimensional rotation matrix, and $\mathbf{t}$ is a translation vector.

The translation is removed by offsetting the centroids of the two meshes to the origin [34]. The rotational component is then estimated using the Kabsch algorithm, which determines the optimal rotation matrix R that minimises the least-squares error between two coordinate sets:

$$R = \arg \min_R \sum_{i=1}^N \|R\mathbf{x}_i - \mathbf{y}_i\|^2 \quad (21)$$

This formulation follows the constrained optimisation approach introduced by [34], where the rotation matrix is required to remain orthogonal. The solution is obtained by constructing a covariance matrix between the two point sets and solving for the rotation that minimises the weighted sum of squared deviations.

The resulting rotation matrix is subsequently converted into yaw, pitch and roll angles to provide an interpretable representation of orientation for downstream control.

This methodology assumes that corresponding points between the reference and extracted geometries are known and consistently ordered, which is satisfied by the structured representation of the extracted meshes. By recovering both position and orientation in this manner, the system can determine the required alignment of embedded components prior to placement.

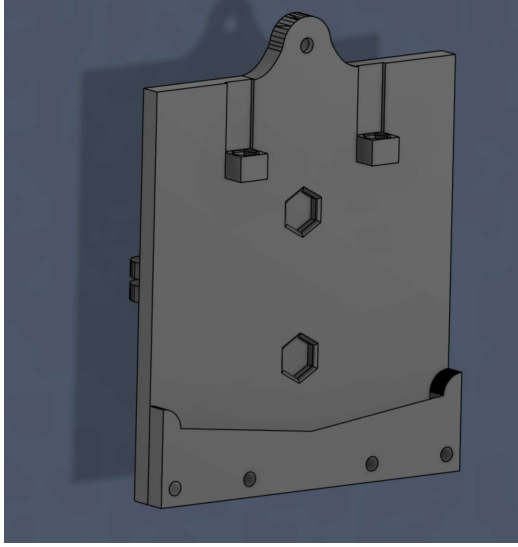
4 Methods, Experimental Setup, and Software Implementation

4.1 Tool-changer

4.1.1 Mechanical Implementation and Assembly

Physical Architecture The implementation is partitioned into three modular assemblies:

- **Modified X-Carriage:** Mounts to the TAZ 5 linear bearings, integrating the load-bearing gravity notch and brass bushings for rotational constraint.
- **Toolhead Backplate:** Interfaces with the extruder assembly, featuring the mating pins and bare circular apertures for gantry engagement.
- **Active Toolchanger Gantry:** An independent frame housing the NEMA 17 motors, dual-rack-and-pinion transmission, and Y-axis docking pins.



(a) Modified X-Carriage



(b) Toolhead Backplate

Figure 10: CAD implementation of the carriage and backplate interface, highlighting the gravity notch and docking apertures.

Material Selection and Additive Manufacturing Structural components were manufactured via FDM using a 20% infill and 3 outer perimeter wall loops to prioritise shell rigidity. Material choices (Table 2) were governed by the thermal requirements of the TAZ 5 print volume and the mechanical demands of the kinematic interface.

Component	Material	Justification
Bushing Housings	PETG	Interlaminar shear strength; $T_g \approx 80^\circ\text{C}$
Gantry Mounts	PETG-CF	Rigidity and resistance to radiant bed heat
Rack-and-Pinion	PETG	Adequate durability for low-velocity actuation

Table 2: Material selection for core toolchanger components.

Assembly and Kinematic Handoff Stainless steel pins and brass bushings were press-fitted into PETG housings to establish the exact-constraint coupling. The active gantry utilises dual MGN12 linear rails to provide zero-backlash translation for the rack-and-pinion mechanism. Custom three-way PETG mounts were utilised to optimise lateral space for toolhead docking while minimising gantry weight.

Mechanical handoff is achieved through vertical Z-axis translation (Figure 11). The gantry extends the docking pins into the toolhead apertures; a subsequent Z-axis descent transfers the gravitational load from the carriage notch to the gantry pins. This vertical decoupling eliminates the high-force dislodging moments associated with lateral breakaway designs.

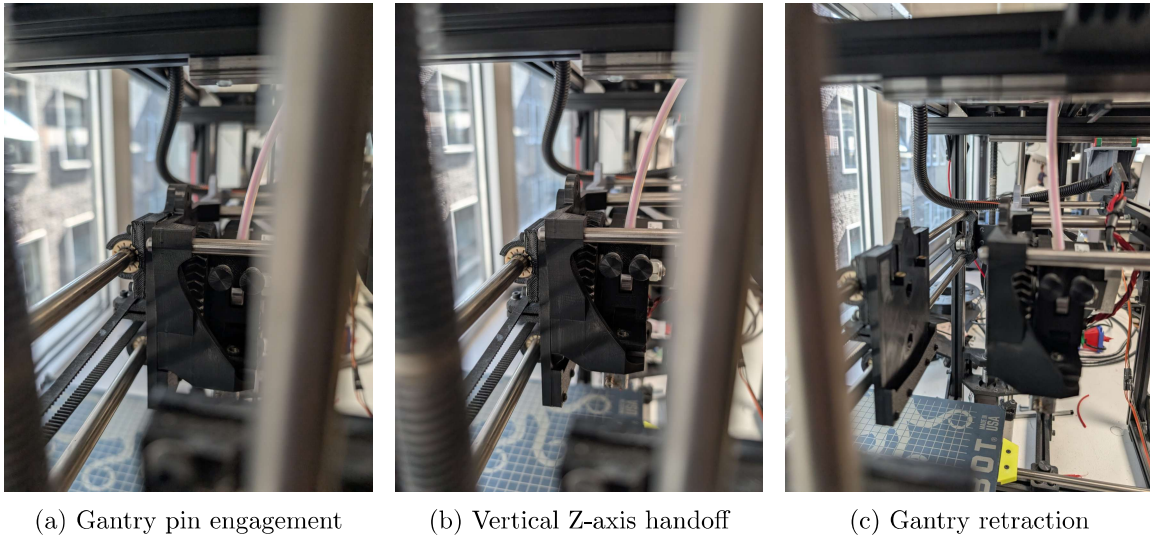


Figure 11: Handoff sequence demonstrating the transfer of the toolhead load to the independent gantry.

4.1.2 Electronics and Firmware Configuration

Electronic Hardware Integration The NEMA 17 gantry motors were controlled via TMC2209 stepper drivers configured in standalone (STEP/DIR) and StealthChop2™ mode to simplify UART communication and minimise audible noise. For the initial prototyping phase, wiring between the steppers, drivers, and the printer’s RAMBo mainboard uses standard jumper wires, with plans to implement sleeved routing for strain relief in future iterations.

To ensure reliable axis homing and precise, consistent movements, and to prevent collisions during the docking sequence, a microswitch endstop was installed at the minimum point of the gantry’s linear rails and wired to the unused Y_MAX endstop pins on the RAMBo board, providing a physical zero reference point for the gantry. It may be important to note that this endstop uses one of the motor extension pins, although no pin conflicts occur in this design.

Power Distribution and Thermal Management During initial testing, power was supplied via an external bench power supply, which offers the key benefit of a current-limiting safety fallback. The active gantry was observed drawing a peak total of 0.5 A at 24 V during actuation, and 0.14 A during idle. Because the tool-changing gantry operates only when the printer’s primary axes and extruders are stationary, this low-power overhead allows the system to safely tap directly into the RAMBo board’s existing 24 V supply rail without overloading the factory power supply as a permanent solution.

The RMS current limit for the gantry stepper motors was physically configured by measuring the reference voltage (V_{ref}) at the potentiometers of the TMC2209 drivers, manually adjusted to approximately 1.15 V. For TMC2209 drivers with standard $0.11\ \Omega$ sense resistors, the RMS current is calculated as $I_{rms} = V_{ref} \times 0.71$, resulting in $I_{rms} \approx 0.81\text{ A}$ and a peak current of approximately 1.15 A, confirmed against the TMC2209 datasheet.

To summarise the electrical characteristics of the active gantry subsystem, the measured power draw and configured current limits are detailed in Table 3.

Table 3: Active Gantry Power Draw and Current Configuration

Parameter	Value
Operating Voltage	24 V
Peak Actuation Current Draw	0.50 A
Idle Current Draw	0.14 A
TMC2209 V_{ref} Setting	1.15 V
Calculated RMS Current (I_{rms})	0.81 A
Calculated Peak Motor Current	1.15 A

To further mitigate heat dissipation at this current, the drivers were fitted with extruded aluminium heatsinks to maintain safe operating temperatures during actuation. $100\ \mu\text{F}$ electrolytic capacitors were utilised for bulk decoupling, smoothing the voltage spikes caused by the driver’s rapid switching behaviour and by back EMF generated in operation.

Subsystem Validation Prior to full integration with the RAMBo mainboard, the gantry’s actuation was validated using an Arduino, and a simple diagnostic script was developed to manually output step, direction, and enable signals to the TMC2209 drivers via user input, and to test homing capabilities and methods. This isolated testing phase verified the smooth mechanical operation of the rack-and-pinion transmission and confirmed the driver tuning without introducing the complexities of the full printer firmware, whilst also mimicking the homing behaviour of slowly reversing until the endstop is reached, and moving forward 5 mm, followed by a slower reversal to obtain a higher accuracy of a zero-point.

Firmware Modification To control the custom tool changer mechanics, the printer’s firmware (running a modified version of Marlin) required customisation to recognise and command the newly integrated axis, which was denoted the U-axis. This was explicitly configured in the firmware architecture as a linear axis rather than the default rotational axis, enabling standard linear motion. It is important to note that, due to limitations in Marlin, linear advance is disabled when additional linear axes are added. However, this feature was unused on the TAZ 5 and hence is a non-issue.

The theoretical linear resolution derived from the gantry kinematics was implemented by updating the firmware’s configuration file to 72.76 steps per millimetre for 1/8 microstepping; however, since 1/16 microstepping is the default on the BTT modules used, the axis steps per mm were calibrated to 36.81.

Furthermore, the axis homing logic was programmed to require that the gantry seek and trigger the physical microswitches before executing any tool-changing sequence. This firmware-level mandate ensures that any accumulated positional error or skipped steps are mechanically zeroed out, guaranteeing that the gantry coordinates remain absolute and repeatable for every handoff. (Note: Specific code modifications to the Marlin configuration

files are detailed in Appendix F.)

4.1.3 Component Tray

The component tray in Figure 12 was also designed to attach to the underside of the bed, placing components at the rear of the printer, and was printed in PETG-CF to handle the bed's heat, with a glass-transition temperature (T_g) of 70°C.

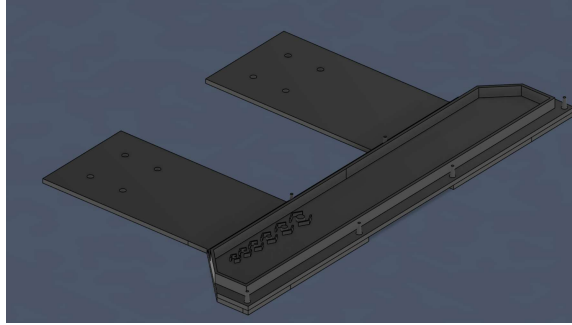


Figure 12: 3D CAD Model of the component tray, with its nut holders and attachment points

4.1.4 Software Logic and Tool-changing Sequence

The tool-change operation follows a sequential state machine, abstracted into external G-code scripts, allowing coordinate adjustments without firmware recompilation. The handoff sequence (Figure 13) utilises vertical Z -axis translation to bypass potential lateral motor stall, ensuring absolute coordinate consistency through mandatory I -axis homing. Blocking M400 commands and absolute positioning (G90) prevent simultaneous axis actuation, maintaining the power-draw constraints of the shared 24V supply rail while mitigating collision risks through 150 mm software limits.

4.1.5 Experimental Setup for Validation

System performance was evaluated across four primary metrics to validate the operational integrity of the tool-changing mechanism:

Validation of Positional Repeatability and Layer Continuity The mechanism must return the nozzle to absolute coordinates with high precision. A continuous extrusion print evaluated mid-print bidirectional swaps to identify visible layer shifting or dimensional discontinuity. Reseating precision was verified by measuring the physical offset between pre- and post-swap perimeters with digital callipers, targeting a sub-0.05 mm variance for high-resolution extrusion compatibility.

Temporal Efficiency/Thermal Stability A Python script logged 50 continuous bidirectional swaps, categorised into eight discrete phases, to evaluate operational impact on part quality. Total execution time was analysed with a focus on the duration of the critical hand-off path. The objective was to verify that tool changes occurred fast enough to keep the printed part above its glass-transition temperature (T_g), preventing thermal contraction and preserving layer adhesion.

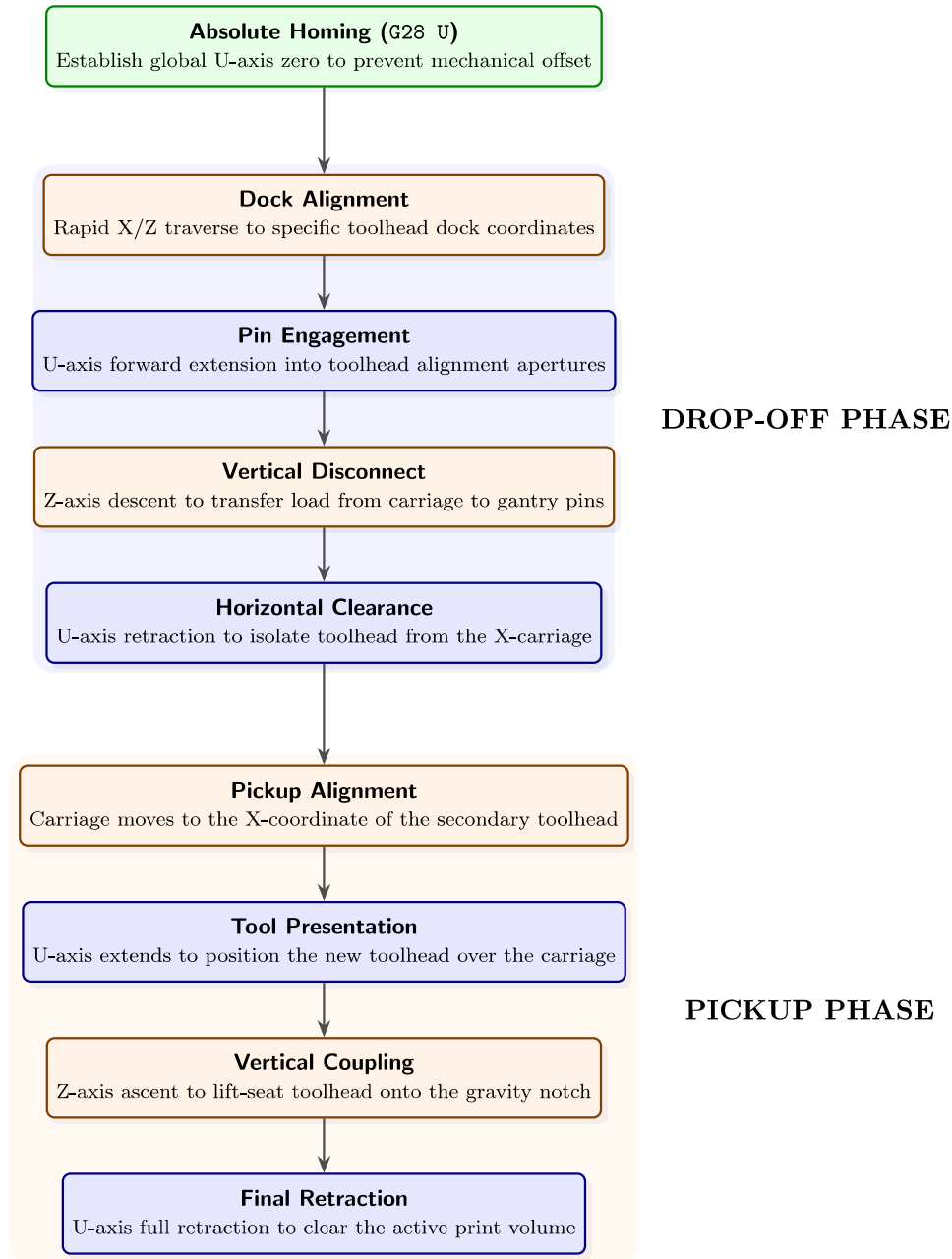


Figure 13: Detailed state machine choreography. Sub-labels delineate the axis movements required to facilitate vertical handoff and avoid motor stall.

Comparative Dynamic Rigidity and Resonance Analysis A comparative resonance test isolated carriage vibrations from systemic frame harmonics. A standard “ringing tower” was printed at 60 mm/s using both the OEM rigid carriage (control) and the modified tool-changer. Analysing ghosting artefacts for peak frequencies and comparing X/Y reverberation patterns validated the structural rigidity of the kinematic coupling against the baseline gantry.

Longitudinal Mechanical Wear Analysis A 50-cycle stress test assessed the durability of the PETG-steel interface. Internal diameters of the bare PETG handoff apertures were measured with digital callipers before and after testing to detect dimensional expansion (sensitivity of ± 0.05 mm). This validated the kinematic sequencing, which transfers gravitational loads via vertical Z-axis translation to eliminate the frictional yielding typical of horizontal sliding phases.

4.1.6 Daughterboard Design

A custom daughterboard (Figure 14) was designed to interface directly with the RAMBo mainboard, consolidating the wiring for the gripper and toolchanger whilst providing dedicated mounts for the TMC2209 stepper drivers. To facilitate I²C communication with the gripper’s AS5600 encoder, the board integrates an AMS1117-3.3 low-dropout (LDO) regulator and a bidirectional logic level-shifting circuit using BSS138 MOSFETs (Figure 15), chosen for their low gate threshold voltage. Additionally, the mainboard’s EXT2 header was broken out to bypass physical obstructions and maintain LCD functionality. The finalised PCB routing and copper layer layouts, prepared for manufacturing, are detailed in Appendix E.

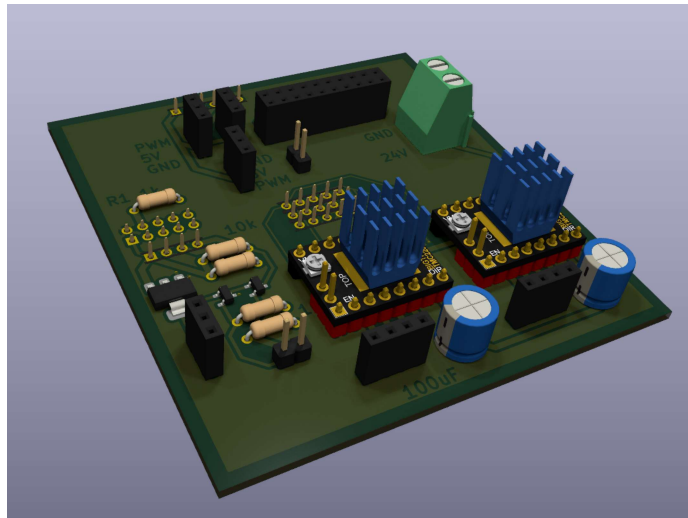


Figure 14: 3D render of the custom interface daughterboard, showing the TMC2209 driver mounts, LDO, and consolidated terminal headers.

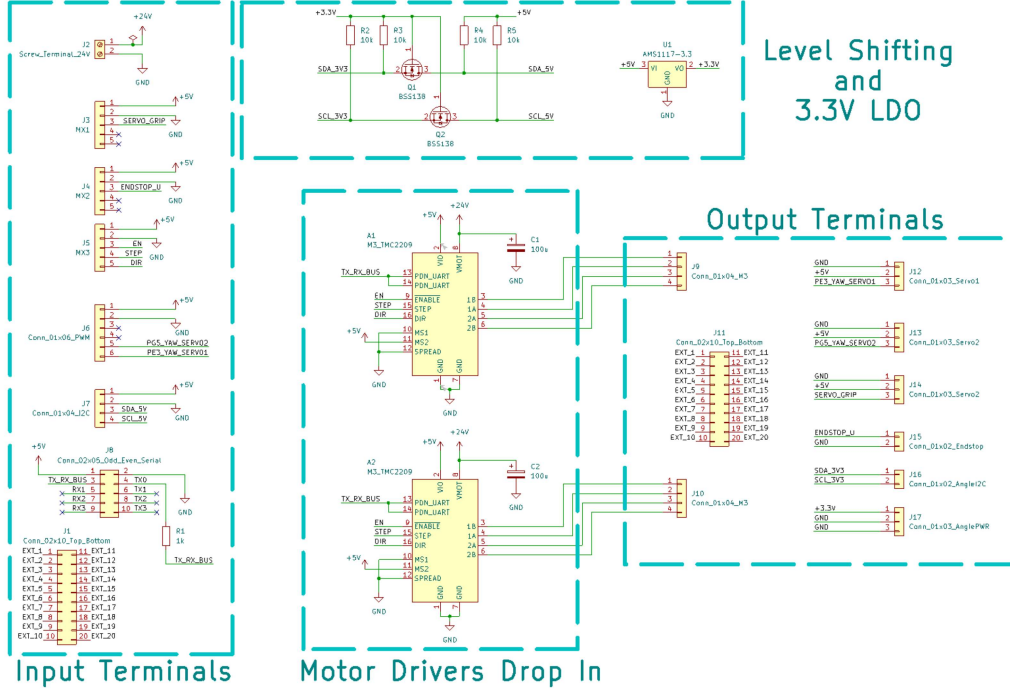


Figure 15: Schematic diagram of the daughterboard, detailing the 3.3V logic level-shifting circuit and stepper driver drop-in integration.

4.2 End-effector System Overview

The end-effector is a servo-driven assembly for pick-and-place of small hardware components on the LulzBot TAZ 5. A parallel two-finger rack-and-pinion gripper was selected over electromagnetic and vacuum alternatives for its self-centring geometry, material independence, and consistent alignment across nut sizes. The system integrates a gripping mechanism, a bevel-spur gear wrist providing yaw rotation, an AS5600 magnetic encoder, and a PID loop for closed-loop wrist positioning.

The 3D-printed gripper mounts directly to the wrist, transferring motion from two SG90 micro-servos via a bevel-and-spur gear train, with the ratio selected to balance torque amplification against the required angular range. The assembly attaches to the toolhead via an interface plate, with servo and encoder wiring routed to an off-carriage Arduino UNO to minimise moving mass, as shown in Fig. 16.

The end-effector uses servo motors for wrist actuation and gripping, with an AS5600 encoder providing wrist angle feedback via I2C to an Arduino PID controller that compensates for backlash, gear compliance, and servo variability. The mechanical and control architecture is detailed in the subsections below.

4.2.1 Mechanical Design and Assembly

The gripper was designed in Fusion 360 and FDM-printed in PLA. A rack-and-pinion mechanism driven by an SG90 micro-servo uses 1.25-module, 24-tooth, 20° pressure-angle gears; the SG90’s 2.5 kg-cm stall torque provides a safety factor of 4 over the estimated 0.5 – 0.65 kg-cm gripping load.

Two iterations were required: the initial 1 mm finger tips were too wide to enter nut

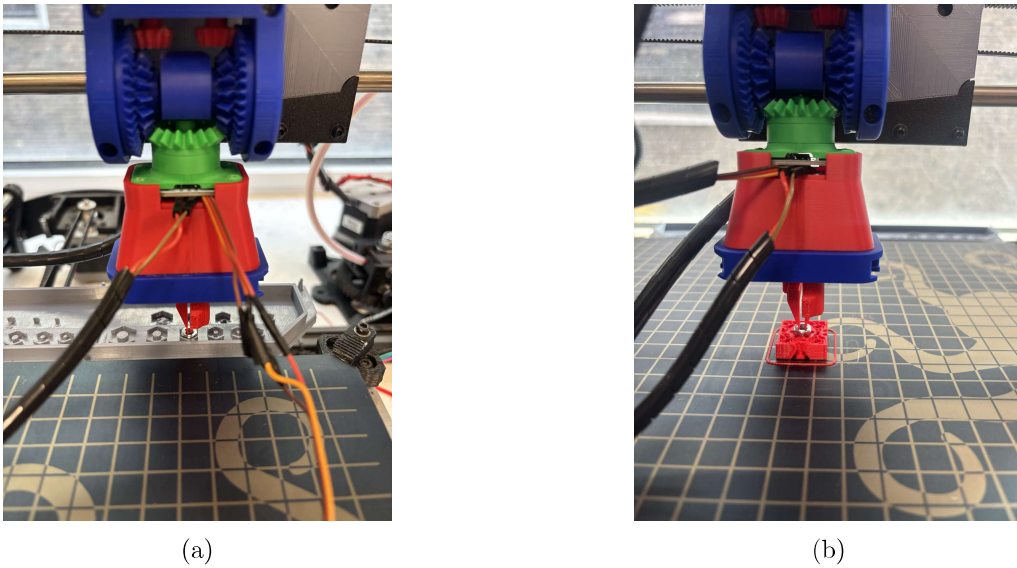


Figure 16: Experimental validation of the end-effector during pick-and-place: (a) gripping a nut from the supply tray, and (b) placing the nut into the target cavity.

cavities, and asymmetric centreline geometry caused workpiece skew. The second design reduced tip thickness to 0.7 mm and corrected the centreline alignment, achieving symmetric closure as shown in Fig. 17.

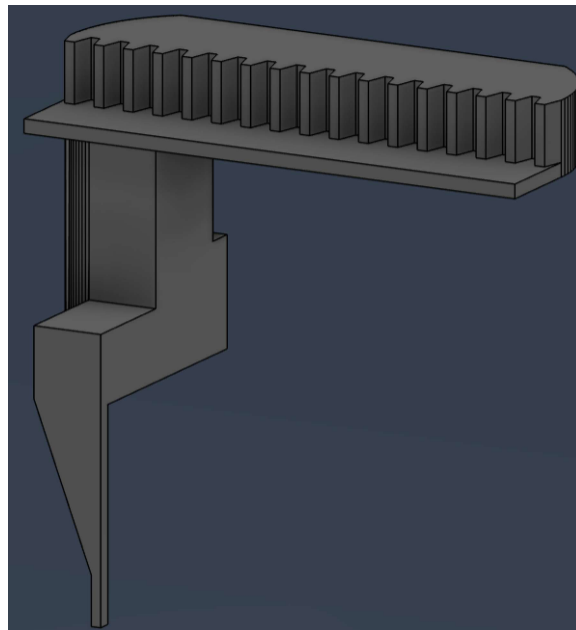


Figure 17: Final gripper finger with a tip thickness of 0.7 mm, showing the rack teeth along the upper surface and the tapered finger body extending downward.

The wrist provides yaw rotation, rotation of the gripper about the vertical axis, to allow correct angular orientation of hexagonal nuts before insertion. It uses a compound bevel gear assembly driven by two SG90 servos, with a central axle (5 mm diameter, 53 mm length) as the primary rotational axis. M3×5 mm threaded brass inserts pressed into the printed housings

provide durable screw engagement at all structural joints, as shown in Fig. 18.

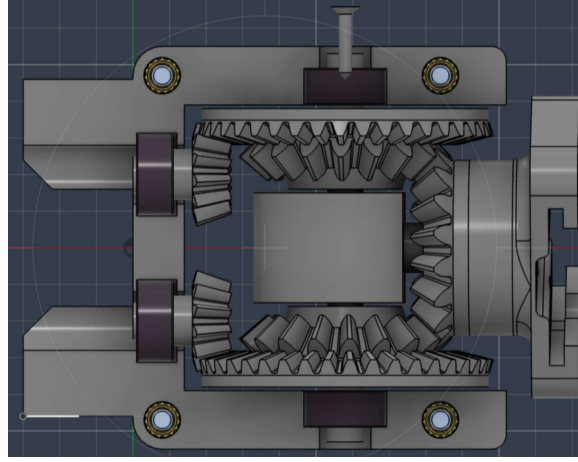


Figure 18: Top-down cross-sectional view of the wrist bevel gear assembly, showing the four meshing bevel gears transmitting rotary motion from the two SG90 servo motors to the central wrist output shaft. The compound gear arrangement converts dual-servo input into a single yaw rotation output, with the gear ratio governing the servo input-to-output angle relationship.

The achievable yaw output angle is governed by the gear ratio of the bevel transmission. For a servo with a nominal 180° range:

$$\theta_{\text{out}} = \frac{180^\circ}{\text{Gear ratio}} \quad (22)$$

where the gear ratio is defined as:

$$\text{Gear ratio} = \frac{N_{LO}}{N_C} \cdot \frac{N_{LI}}{N_S} \quad (23)$$

and N_{LI} , N_{LO} are the inner and outer tooth counts of the compound bevel gear, N_S is the small bevel gear tooth count, and N_C is the central bevel gear tooth count. In the initial design, $N_{LI} = 18$, $N_S = 12$, $N_{LO} = 40$, and $N_C = 18$, giving:

$$\text{Gear ratio} = \frac{40}{12} \cdot \frac{18}{18} = 3.33 \quad (24)$$

$$\theta_{\text{out}} = \frac{180^\circ}{3.33} = 54^\circ \quad (25)$$

A hexagonal nut has sixfold rotational symmetry, requiring a minimum wrist output range of 60° to cover all unique orientational states. The initial design fell 6° short. Rather than modifying the outer gear stage, which would have required redesigning the wrist frame and axle geometry, the inner tooth count was reduced to $N_{LI} = 16$, giving:

$$\text{Gear ratio} = \frac{40}{12} \cdot \frac{16}{18} = 2.96 \quad (26)$$

$$\theta_{\text{out}} = \frac{180^\circ}{2.96} = 60.8^\circ \quad (27)$$

This confirmed that the revised design achieves a wrist output range marginally above the 60° minimum, resolving the angular shortfall.

4.2.2 Sensing and Feedback Implementation

The AS5600 magnetic rotary encoder was selected for absolute, contactless angle measurement with 12-bit resolution ($0.08^\circ/\text{count}$) over I2C, eliminating the homing sequence that would otherwise add latency to each pick-and-place cycle.

The magnet is fixed to the rotating wrist shaft while the sensor IC mounts on the stationary base plate beneath it, connected to the Arduino via I2C. As the wrist rotates, the AS5600's internal CORDIC processor converts Hall-element readings into a 12-bit angle value, as shown in Fig. 19.

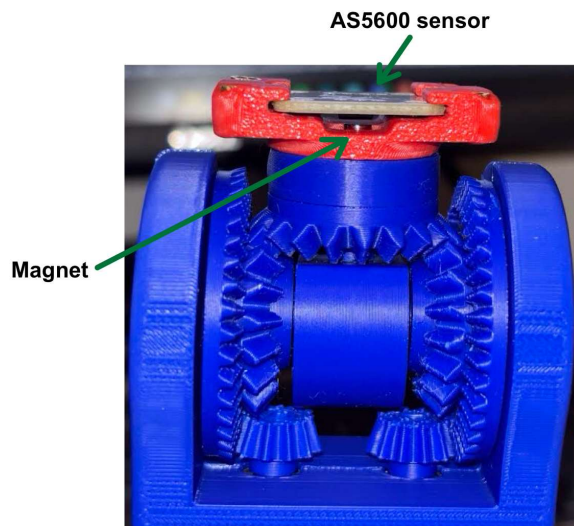


Figure 19: Placement of the AS5600 sensor and magnet within the wrist assembly. The magnet is mounted on the rotating shaft above the sensor, allowing contactless measurement of wrist angle through changes in magnetic field orientation.

The sensor holder was designed to maintain an air gap of $1.0 - 1.5$ mm, verified via the `STATUS` register. A printed cylindrical pocket maintains magnetic alignment to meet the air-gap requirement.

Once integrated, each motor was swept across its PWM range, and the recorded encoder angles were processed in MATLAB to obtain per-motor calibration functions.

4.2.3 Control System Implementation

The control system regulates the angular position of the wrist joint using real-time feedback from the AS5600 encoder.

The closed-loop PID control structure follows a standard error-correction feedback loop. A desired wrist angle θ_{ref} is compared to the measured angle θ_{meas} from the AS5600. The error:

$$e(t) = \theta_{ref} - \theta_{meas} \quad (28)$$

is processed by a PID controller to generate a control output $u(t)$, shown in Eq. 9.

This is mapped to a PWM command for the servo motors via the motor-specific calibration function. The block diagram of this control structure is shown in Fig. 20.

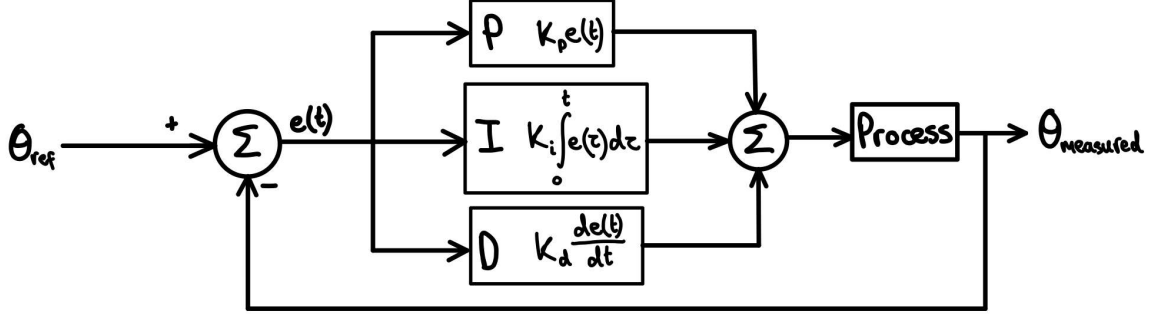


Figure 20: Block diagram of the PID controller having as input the reference angle we desire, and the output angle we measured through the angle sensor.

4.2.4 Controller Tuning

Analytical tuning methods were inapplicable due to backlash. Manual iterative tuning was used instead, shown in Appendix 9.4, with step responses observed via the Arduino Serial Plotter:

1. $K_p = 0.3$ provided responsive correction without exciting mechanical resonance of the gear train.
2. $K_i = 0.4$ eliminated residual steady-state error caused by the backlash dead zone, kept moderate to avoid overshoot during integrator accumulation.
3. $K_d = 0.001$ damped oscillatory behaviour at the gear mesh compliance frequency, kept small to avoid amplifying encoder measurement noise.

The reference angle was ramped incrementally in firmware rather than applied as a step discontinuity, limiting peak gear loading during large-angle commands. The tuned gains produced a monotonically convergent, non-oscillatory step response, with performance characterised in Section 5.

4.2.5 Wrist Servo Motor Calibration

Calibration was necessary because SG90 hobby servo motors exhibit motor-to-motor variability in their pulse-width-to-angle characteristics, and the geared wrist transmission further modifies the relationship between motor input and joint output. Each motor was therefore characterised independently using the AS5600 encoder. PWM pulse widths were swept from $500 \mu\text{s}$ to $2300 \mu\text{s}$ in steps of $200 \mu\text{s}$, with the encoder angle recorded at each step. Data above $2300 \mu\text{s}$ were excluded as the servos had reached their mechanical endpoint. A linear polynomial was fitted to each dataset in MATLAB using `polyfit`, giving the calibration functions shown in Fig. 21:

$$\text{Servo}_1(\text{PWM}) = 0.013198(\text{PWM} - 500) - 0.299414 \quad (29)$$

$$\text{Servo}_2(\text{PWM}) = 0.013278(\text{PWM} - 500) - 0.589778 \quad (30)$$

where PWM is in microseconds and the output is the wrist angle in degrees.

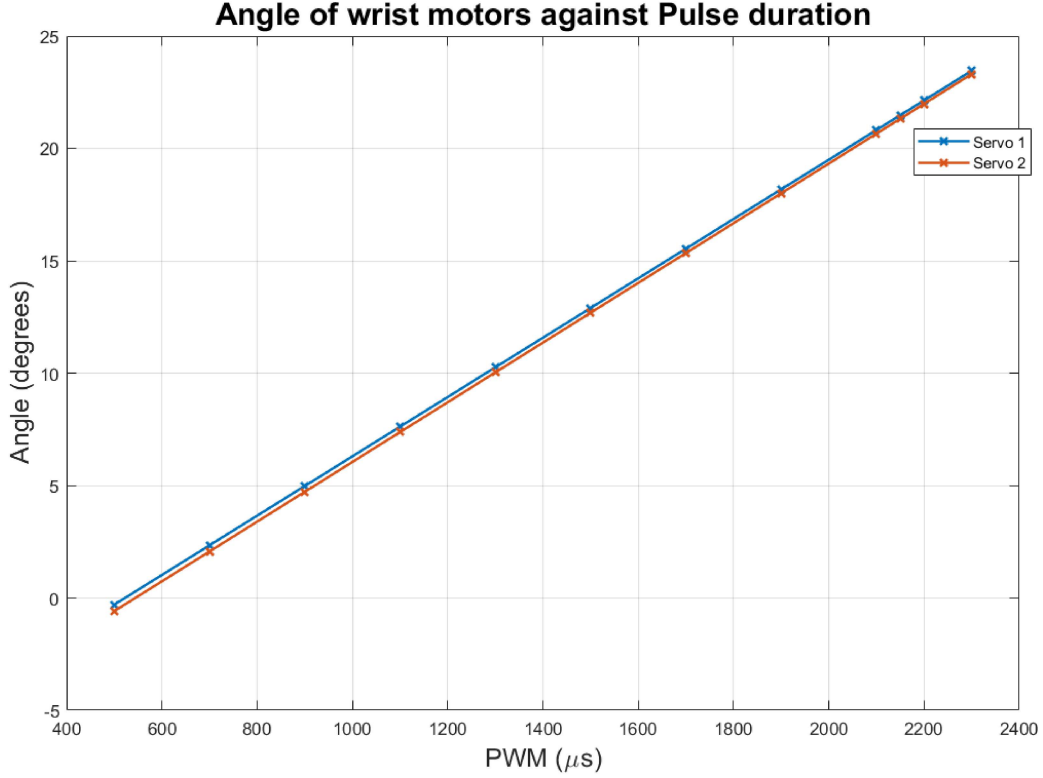


Figure 21: Calibration curves for the two wrist servo motors, showing measured wrist output angle as a function of commanded PWM input. Both responses are approximately linear across the operating range, with small deviations justifying the use of separate interpolation-based calibration functions for each motor.

The two slopes differ by $0.000080 \text{ deg}/\mu\text{s}$, a relative discrepancy of approximately 0.6%, producing a cumulative angular divergence of 0.14° over the full $1800 \mu\text{s}$ active range. The offset terms also differ (-0.299° versus -0.590°), reflecting assembly tolerance in each motor's zero-position. These differences confirm that individual calibration functions are necessary rather than a single shared mapping.

Gripper actuation was handled separately using a hard-coded lookup table, since it operates at a finite number of repeatable positions rather than requiring continuous regulation. Discrete open and close angles were determined experimentally for each nut size and stored as constants in the Arduino firmware, as summarised in Table 4.

Table 4: Gripper servo angles for each tested nut size, determined experimentally and stored as constants in the Arduino firmware.

Nut Size	Open Angle (°)	Close Angle (°)
M2	146	180
M2.5	151	180
M3.5	158	180
M4	160	180
M5	166	180
M6	172	180

4.2.6 Software Architecture

The end-effector control was implemented on an Arduino UNO, structured around four sequential stages: serial command reception, encoder angle acquisition, PID computation, and servo actuation. This is depicted in Appendix 9.4.

Commands were received via UART at 9600 baud. Wrist rotation commands specified a target angle in degrees. Gripper commands specified an open or close action with a nut size identifier. This kept closed-loop control active only where required, wrist positioning, while gripper actuation remained a direct open-loop lookup.

The AS5600 was addressed over I2C at slave address 0x36. The filtered ANGLE register pair (0x0E high byte, 0x0F low byte) was used in preference to the RAW ANGLE registers (0x0C/0x0D), as the sensor’s internal moving-average filter reduces measurement noise without meaningful latency at the 50 ms loop period. The two bytes are combined into a 12-bit integer:

$$\text{raw} = (\text{reg}_{0x0E} \ll 8) \mid \text{reg}_{0x0F} \quad (31)$$

and converted to degrees by scaling across the full 4096-count range:

$$\theta_{\text{deg}} = \text{raw} \times \frac{360}{4096} \quad (32)$$

giving a resolution of 0.0879° per count [18]. A one-time offset θ_{offset} , recorded at the wrist’s mechanical zero during initialisation, was subtracted to obtain a repeatable positional reference:

$$\theta_{\text{wrist}} = \theta_{\text{deg}} - \theta_{\text{offset}} \quad (33)$$

The PID loop executed at 50 ms per iteration, ramping the reference angle at 1° per iteration to limit peak gear loading. A serial “OK” acknowledgement was returned only once the measured angle had remained within $\pm 0.5^\circ$ of the target for five consecutive iterations. Gripper commands were acknowledged immediately, as no position feedback was required.

4.3 Machine Vision System Implementation

4.3.1 System Design and Hardware Selection

The design of the machine vision system was guided by the requirements outlined in Section 2.3.1, with emphasis on achieving reliable classification accuracy while maintaining low cost and compatibility with embedded hardware. The system was developed as a lightweight processing pipeline for integration with a Raspberry Pi platform.

The Raspberry Pi 3B+ was selected as the primary processing unit due to its availability. While more powerful platforms exist, it provides sufficient computational capability for embedded inference. However, its limited processing power and memory constrain model complexity and image resolution, influencing design decisions.

Various camera solutions were evaluated during the design phase, as discussed in Section 2.3.2. Based on this evaluation, a global shutter camera was selected to ensure consistent image acquisition and minimise motion-induced distortion. The Raspberry Pi Global Shutter Camera was coupled with a C-mount varifocal lens (2.8–12 mm), as shown in Figure 22. The focal length was selected based on the required working distance and field of view, ensuring the nut occupies a sufficient portion of the image for feature extraction while retaining surrounding context for accurate classification. This balance between spatial resolution and contextual information was critical to achieving the required classification accuracy.



Figure 22: C-mount 12 mm Arducam lens [35] used with the Raspberry Pi Global Shutter Camera, selected to achieve the required field of view and working distance for accurate feature capture.

Practical integration constraints were addressed. The camera module was supplied with a short ribbon cable, which required the Raspberry Pi to be positioned close to the camera, limiting layout flexibility. To overcome this, a longer 1 m cable was used, enabling more flexible positioning of the Raspberry Pi relative to the printer without compromising connectivity.

4.3.2 Experimental Setup

The experimental setup consisted of a Raspberry Pi 3B+ connected to a global shutter camera via a Camera Serial Interface (CSI) ribbon cable. The camera was mounted perpendicular to the working surface at approximately 90°, providing a fixed overhead view and ensuring consistent image capture across all experiments. As shown in Figure 23, the camera was positioned above the printer bed, capturing the region where nut placement occurs. This fixed positioning ensures consistent geometric and lighting conditions, which are critical for reliable machine vision performance.

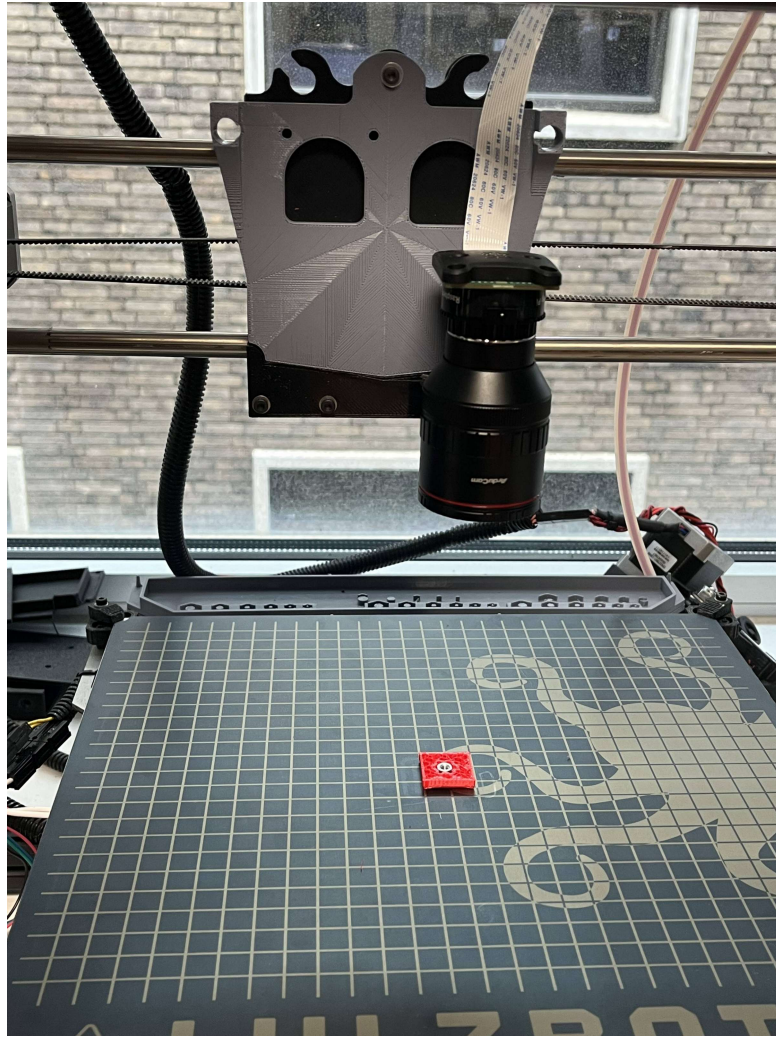


Figure 23: Experimental setup showing the overhead global shutter camera mounted above the bed of the printer.

The Raspberry Pi was powered via an external supply and connected to a display via HDMI for real-time monitoring. Input devices were connected via USB for direct interaction during development and testing.

For automated operation, the Raspberry Pi was connected to a laptop via Ethernet. The laptop sends a trigger signal for image capture and classification, and the Raspberry Pi returns the classification result, where a value of 1 indicates success and 0 indicates failure. This communication enables integration with the wider automation system.

4.3.3 Dataset Collection and Preparation

A dataset was collected to train a binary classification model to distinguish between successful and failed nut placements. Four nut sizes (M3.5, M4, M5, and M6) were included to ensure generalisation across scale variations. For each size, multiple cavity positions were used to prevent overfitting to a fixed spatial location, as illustrated in Figure 24.

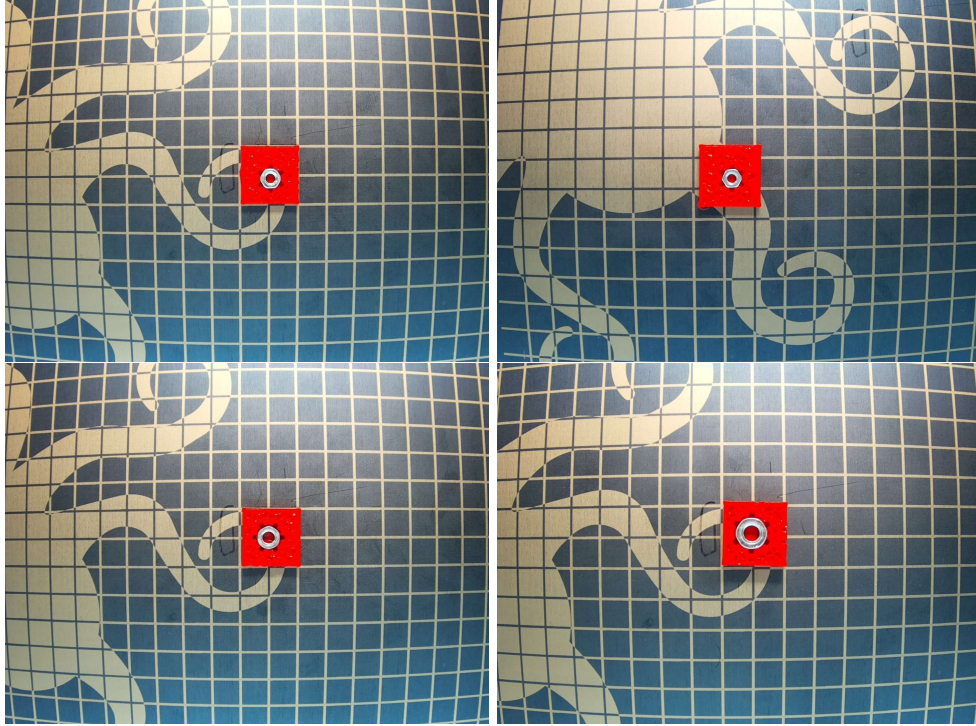


Figure 24: Examples of successful nut placements across different sizes: (top left) M3.5, (top right) M4, (bottom left) M5, and (bottom right) M6.

A successful placement was defined as a nut seated flush within the cavity with correct alignment, while failure cases included misaligned, tilted, or partially inserted configurations. Failure scenarios were selected to reflect realistic operational errors rather than arbitrary placements, as shown in Figure 25.

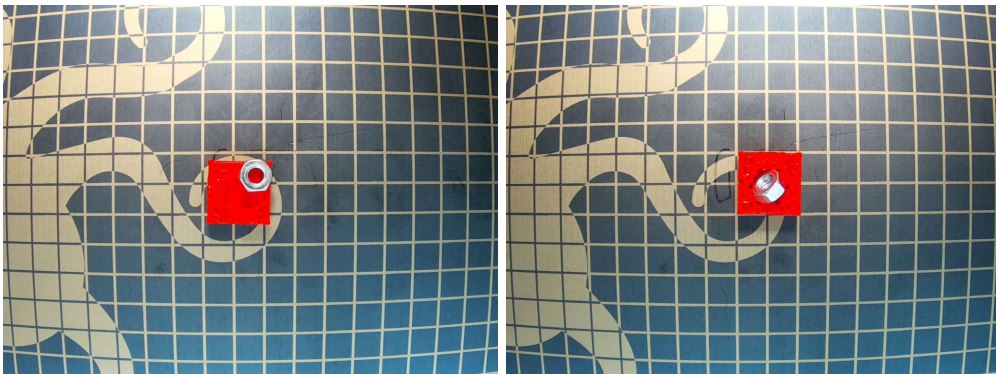


Figure 25: Examples of realistic failure cases: (left) misaligned nut, (right) partially inserted nut.

Initial data collection consisted of 50 images per class for each nut size, resulting in a balanced dataset of approximately 400 images. Variation was introduced during data collection, including changes in lighting conditions, camera height, cavity positioning on the bed, and orientation, to improve generalisation.

Following initial training, misclassified and low-confidence cases were identified, and additional images of these scenarios were collected. Particular focus was given to visually ambiguous configurations, such as diagonally misaligned nuts, as shown in Figure 26. This iterative process improved performance on challenging cases while maintaining class balance. The final dataset comprised approximately 500 images.

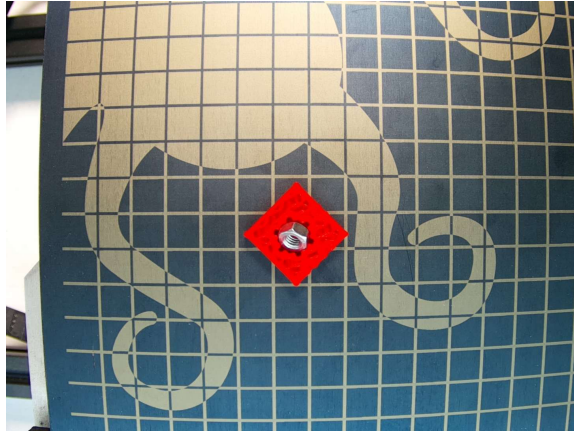


Figure 26: Example of a visually ambiguous failure case, where similar visual features can lead to misclassification by the model.

4.3.4 Model Architecture and Training Strategy

The classification system was implemented using a CNN in Python with TensorFlow and Keras. A transfer learning approach, as described in Section 2.3.3, was adopted using the MobileNetV2 architecture pre-trained on the ImageNet dataset. ImageNet is a large-scale dataset containing millions of labelled images across diverse categories, enabling models to learn general visual features such as edges, textures, and shapes. Leveraging these features reduces the amount of task-specific data required and improves convergence speed.

MobileNetV2 was selected due to its computational efficiency and suitability for embedded systems. It is a lightweight CNN designed to reduce computational cost while maintaining strong performance, making it well-suited for deployment on resource-constrained hardware such as the Raspberry Pi 3B+. This enables efficient inference while maintaining sufficient accuracy for reliable classification. The overall architecture of the classification pipeline is illustrated in Figure 27.

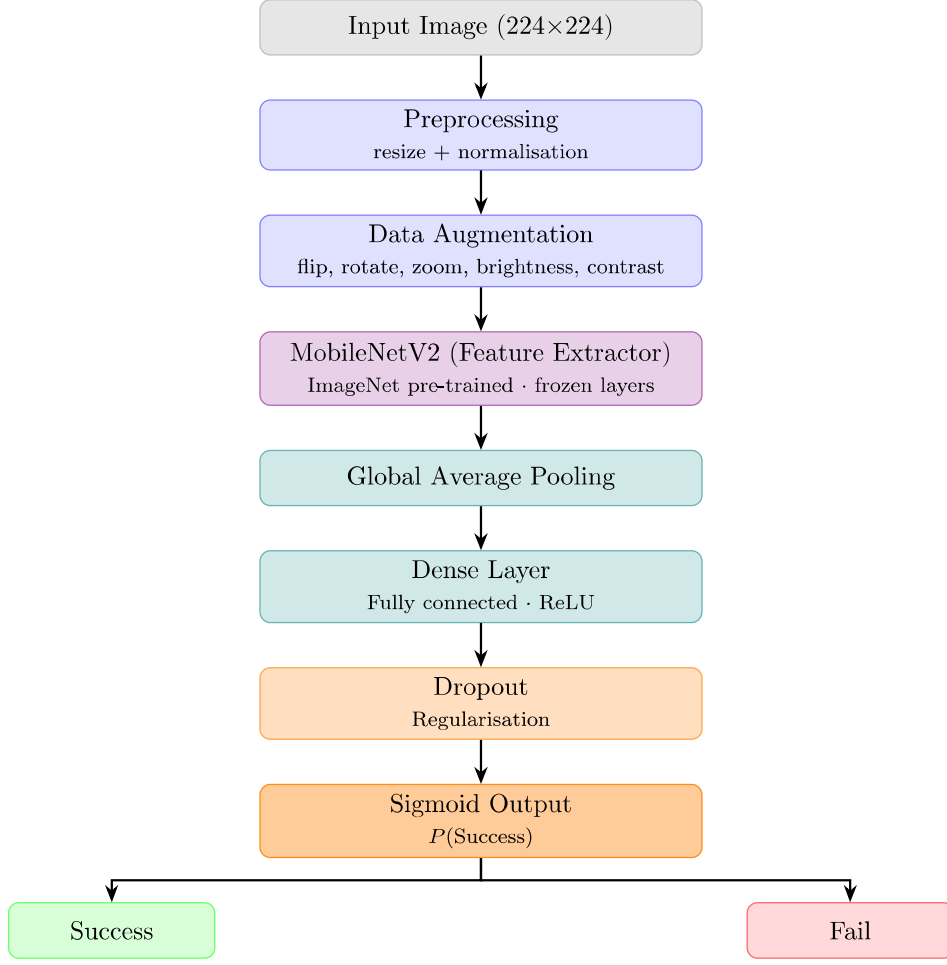


Figure 27: Schematic of the CNN architecture, illustrating preprocessing, data augmentation, feature extraction using a pre-trained MobileNetV2 backbone, and the final classification layers.

Input images are resized to 224×224 pixels to match the expected input dimensions of MobileNetV2, and pixel values are normalised to the range $[0, 1]$ before being passed through the network. The pre-trained network was used as a feature extractor, with most layers frozen to retain general visual features, while the final layers were fine-tuned for nut placement classification.

To improve generalisation and reduce overfitting, data augmentation was applied during training, including random flipping, rotation, zoom, brightness, and contrast adjustments. This increases data diversity and exposes the model to variations in orientation and lighting conditions, improving robustness to real-world scenarios.

The MobileNetV2 feature extractor outputs a spatial feature map, which is reduced to a fixed-length vector via Global Average Pooling, removing the need for flattening and reducing the number of parameters passed to subsequent layers. This is followed by a fully connected Dense layer with ReLU activation, which learns task-specific feature combinations. A Dropout layer is applied during training as a regularisation mechanism, randomly deactivating neurons to prevent co-adaptation and improve generalisation. Finally, a single neuron with a Sigmoid activation function produces a probability score in the range $[0, 1]$, which is thresholded to classify the placement as *Success* or *Fail*.

The model was trained using binary cross-entropy loss and the Adam optimiser. Class weighting was applied to prioritise correct identification of failure cases, reflecting the asymmetric cost of misclassification in a quality control context. Early stopping was used to prevent overfitting by halting training when validation performance no longer improved. The full implementation of the training and evaluation pipeline is provided in Appendix 9.1.

4.3.5 Model Deployment

Following training, the model was exported and converted to TensorFlow Lite (TFLite) format to enable efficient deployment on the Raspberry Pi 3B+. TensorFlow Lite is a lightweight inference framework designed for resource-constrained devices, reducing model size and memory footprint while maintaining predictive performance.

The Raspberry Pi 3B+ is equipped with a 1.4 GHz quad-core ARM Cortex-A53 CPU and 1 GB of RAM, limiting its ability to efficiently run standard TensorFlow models. As a result, model optimisation and conversion to TFLite were necessary to achieve practical inference speeds and reliable real-time operation. This is important as classification must be completed within a short time window, allowing the printer's extruder to remain at operating temperature and ensuring a smooth transition between operations without repeated heating cycles.

Inference is performed using the TFLite interpreter, which loads the converted model and processes each input image through the same preprocessing pipeline used during training, ensuring consistency between training and deployment.

4.3.6 Evaluation and Inference

Model performance was evaluated on a validation set, with predicted labels, ground truth, confidence scores, and classification correctness recorded for each image, as discussed in Section 5.3.1. This enabled systematic identification of challenging edge cases, particularly subtle misalignment scenarios where the visual difference between correct and incorrect placement is minimal.

An inference pipeline was implemented for real-time testing on unseen images. Images are preprocessed, passed through the trained model, and classified based on the defined threshold, reflecting the intended deployment scenario. A decision threshold of 0.75 was used instead of the default 0.5 to reduce false positives and ensure that a placement is only classified as *Success* when the model exhibits sufficiently high confidence.

4.4 Software and Firmware Implementation

4.4.1 CAD Preparation and Embedded Component Extraction

Figure 28 illustrates the workflow used to convert the user-input CAD model into executable G-code instructions. In contrast to the conventional pipeline, where the CAD model is passed directly to the slicer, this approach introduces an intermediate extraction stage. This stage separates the embedded components from the printable geometry and generates component metadata used to coordinate the embedding process during printing. As a result, the extraction stage produces both a print-only model for slicing and a set of component parameters for runtime control.

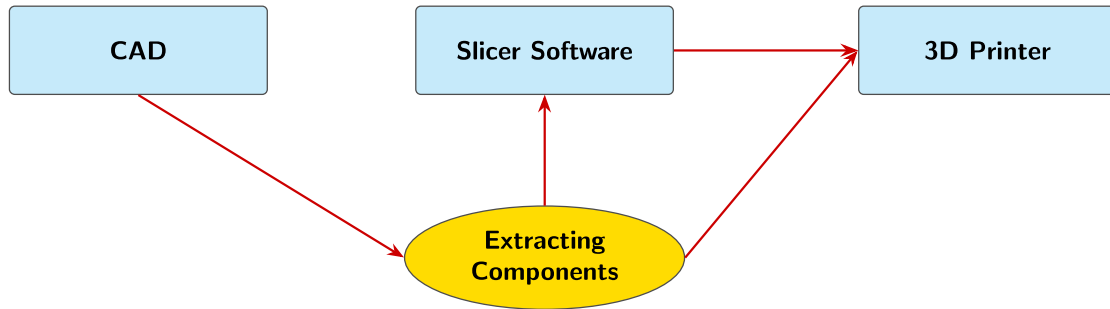
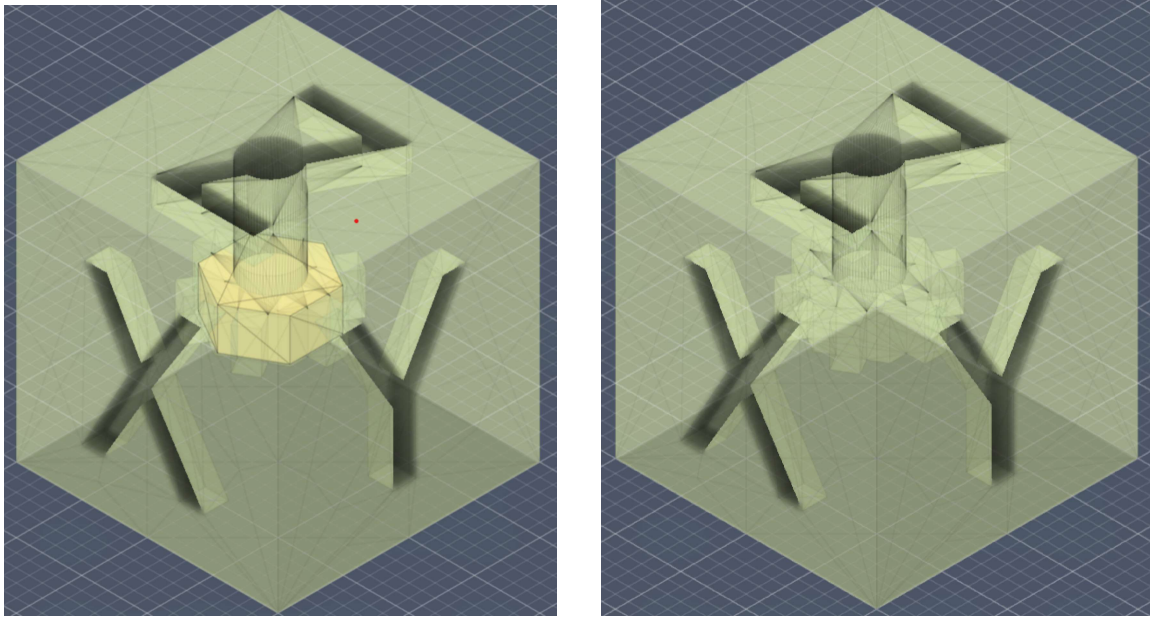


Figure 28: Modified CAD-to-print workflow. The blue rectangles represent the stages of the conventional 3D printing sequence, while the yellow stage represents the additional script introduced in this project. The extraction script separates the printable model from the embedded components and provides component data for use during printer execution.

The input CAD model must therefore contain both the object to be printed and the components to be embedded, as illustrated in Figure 29a. To enable correct fabrication, these elements must be isolated prior to slicing, since all geometry present in the CAD model is typically interpreted by the slicer as material to be printed.



(a) Original CAD model containing both the printable geometry and embedded component.

(b) CAD model after extraction, containing only the printable geometry with cavity.

Figure 29: CAD models before and after processing by the extraction script, viewed in Autodesk Fusion. The extraction stage removes embedded components prior to slicing, leaving cavities for later insertion during the printing process. The cavity geometry consists of a hexagonal recess sized with a 0.1 mm clearance relative to the nut, along with a cylindrical through-hole extending to the surface of the part to allow a bolt to be fastened to the embedded nut.

The 3MF file format was selected as the required input, as it supports the storage of multiple objects within a single file while maintaining a shared coordinate system. Within the model, components intended for embedding were identified using a naming convention, where each embedded object was prefixed with **Embed-**. This enabled automated identification of embedded components without requiring additional metadata. Autodesk Fusion was used as the CAD software, as it preserves object names within the exported 3MF file, allowing the naming convention to be reliably utilised.

The exported 3MF file was then processed using a custom Python extraction script, which is provided in Appendix B. As the 3MF format is internally structured as a compressed archive containing XML-based geometry data, the script reads the model file and identifies all objects whose names follow the **Embed-** naming convention, corresponding to embedded components. These objects are removed from the file, producing a modified 3MF file containing only the geometry to be printed. This ensures that the slicer generates toolpaths that form cavities for the embedded components, as seen in Figure 29b.

In parallel, geometric information for each embedded component is extracted and stored in a structured JSON file. This includes the component name, mid-point position and the minimum and maximum Z-coordinates of the component mesh. The mid-point position defines the target placement location, while the vertical bounds are used to determine the appropriate layer at which embedding should occur during printing. In addition to positional data, the orientation of each component is determined by comparing the extracted mesh with a corresponding reference mesh from a predefined component library. The reference mesh represents

the nominal pickup configuration of the component. A rigid transformation is computed between the reference and extracted geometries using the Kabsch algorithm introduced in Section 3.3. The resulting rotation is expressed in terms of yaw, pitch, and roll, allowing the required placement orientation to be derived directly from the CAD model without manual specification.

Following extraction, the print model is translated such that it is centred on the printer bed, which is repeated for the extracted components.

4.4.2 Slicer Profile Design

OrcaSlicer was selected as the slicing software for this project because it provides a default profile for the LulzBot TAZ 5, allowing the printer configuration to be established from a known baseline. This profile was used as the baseline configuration, with targeted modifications introduced to reliably incorporate the embedding process. The retraction distance was reduced from 5 mm to 1 mm because the default value was found to reduce extrusion consistency after short pauses, such as layer transitions and embedding interruptions. This helped maintain filament pressure in the hot end and improved extrusion reliability during start-stop operation.

A custom purge sequence consisting of two straight extrusion lines was also added at the start of the print to prime the nozzle before the main geometry. A prime tower was used for post-embedding purging, as described in Section 4.4.3.

In addition to extrusion-related modifications, the sparse infill pattern was changed from the default infill to a gyroid structure. This decision was informed by the requirements of the machine vision system. The gyroid pattern provides a visually distinct texture compared to the monotonic internal solid infill, as shown in Figure 30, as well as the solid hexagonal shape of the embedded nut. This improves the ability of the camera-based system to differentiate between printed material and embedded components, thereby enhancing its reliability.

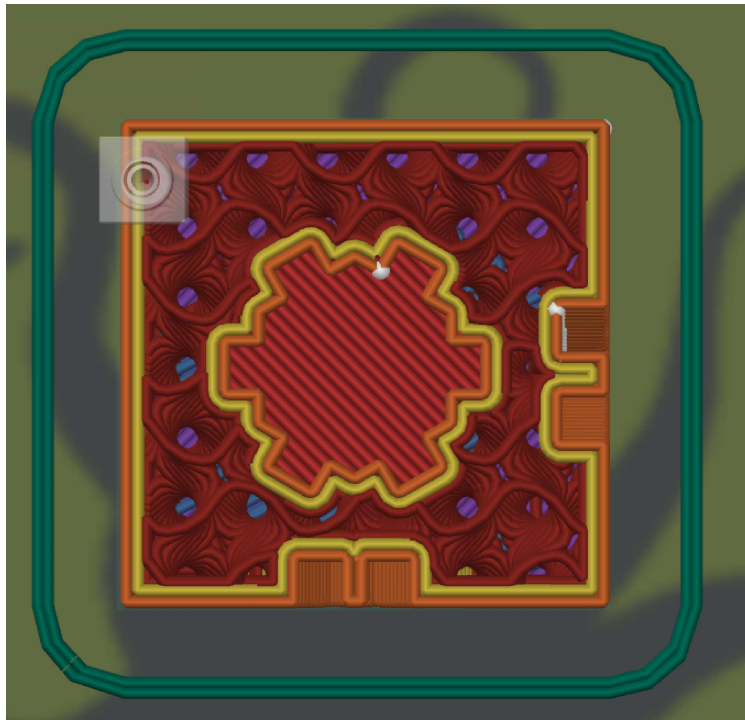


Figure 30: OrcaSlicer preview showing the gyroid sparse infill (red) pattern used in the print. This pattern is visually distinct from the monotonic internal solid fill (orange), which aids the machine vision system in differentiating between printed material and the embedded component and cavity geometry.

Overall, the slicer profile remained close to the default TAZ 5 configuration, with modifications limited to parameters that directly influenced extrusion stability, embedding robustness and machine vision performance. This ensured that the printer operated within a trusted regime while accommodating the additional requirements of the embedding process.

4.4.3 G-code Streaming and Embedding Implementation

Following slicing, the generated G-code is processed through a custom Python-based streaming system, which is given in Appendix C, rather than being uploaded directly to the printer. This enables the print to be interrupted at selected layers so that components can be inserted after their cavities are formed but before they are sealed.

The overall execution workflow of the script is illustrated in Figure 31. First, the G-code file is analysed to determine the layer height. The embedding layer for each component is then computed by dividing the component's maximum Z-coordinate by the layer height. The floor value is taken to ensure that embedding occurs before the printer begins to seal the cavity. The G-code file is read sequentially and transmitted to the printer via a serial communication interface. The G-code is transmitted line-by-line over serial, with firmware acknowledgements and command timeouts used to maintain synchronisation and prevent indefinite blocking.

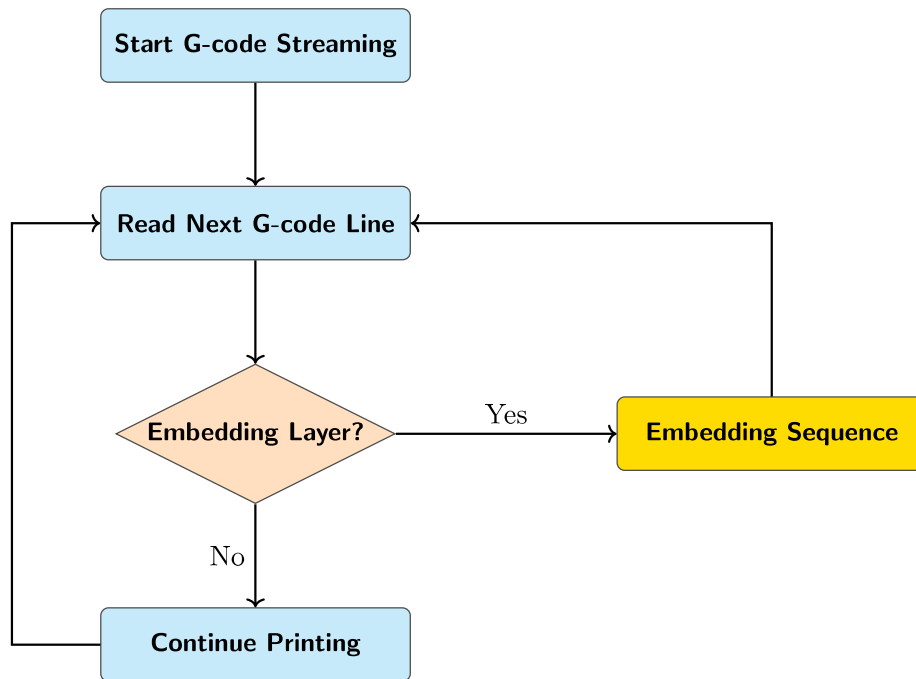


Figure 31: High-level printing workflow during G-code streaming. At the start of each layer, the code detects if the current layer corresponds to one of the embedding layers. If so, the embedding sequence will be triggered whereas the standard printing process will be carried out if not.

When an embedding event is triggered, the current print position is first preserved to allow accurate resumption after the sequence. The extruder temperature is also lowered to 180°C to minimise leakage while the embedding process occurs. The print head is then lifted along the positive Z-axis by 15 mm to ensure a sufficient clearance before initiating the embedding routine. This transition is handled through predefined G-code commands and interfacing with the auxiliary systems as follows:

1. The tool-change system exchanges the print head with the gripper.
2. The gripper collects the component from the predefined pick-up location.
3. The gripper moves to the extracted CAD target position and releases the component into the cavity.
4. The camera is positioned for inspection and the machine-vision system verifies the placement.
5. If placement is successful, the gripper is exchanged back to the print head. Otherwise, the print is terminated.

The sequence coordinates the printer firmware, tool-change mechanism, gripper and machine-vision module using serial communication to the RAMBo board and Ethernet messaging to the Raspberry Pi 3B+.

A gap was observed between the embedded layer and the subsequent layer, as shown in Figure 32. This was attributed to cooling of the previous layer during the embedding pause and loss of nozzle pressure during idling. To mitigate this, the nozzle is set to 205°C and moved over the previous layer without extrusion to partially remelt the surface and improve adhesion. A purge sequence is then carried out using the slicer-generated prime tower, restoring nozzle